



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



Estructura de datos con memoria dinámica

Las estructuras de datos con memoria dinámica se usan para **optimizar** el uso de **memoria** cuando **almacenamos** datos

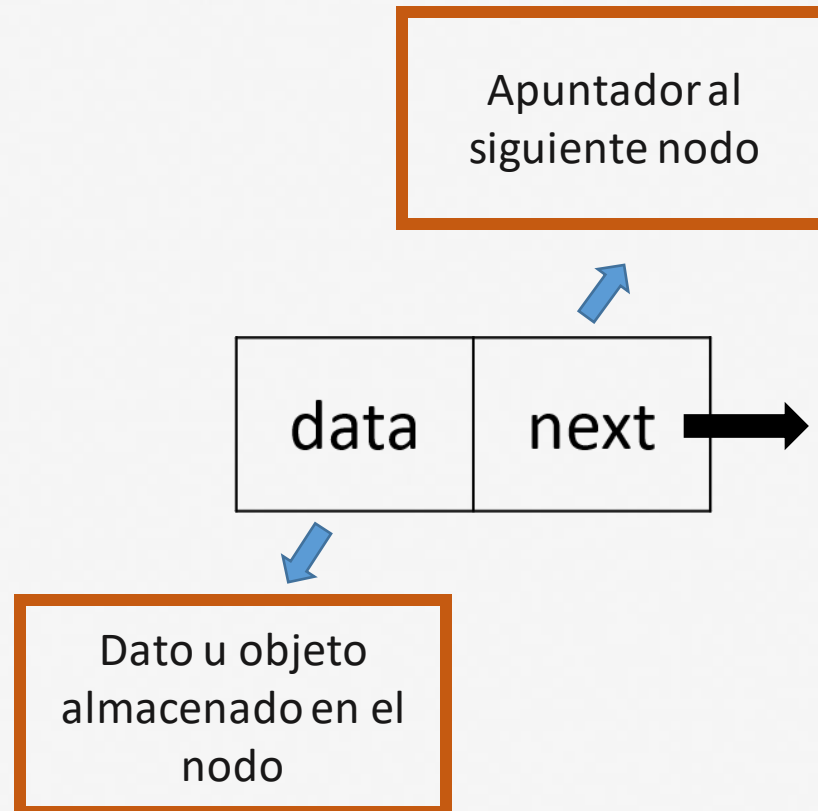
Estas estructuras siguen siendo **colecciones** de datos, pero con formas de acceso **diferentes**

Las estructuras de datos de memoria dinámica son:

- Listas enlazadas simples
- Listas enlazadas dobles
- Pilas
- Colas
- Árboles

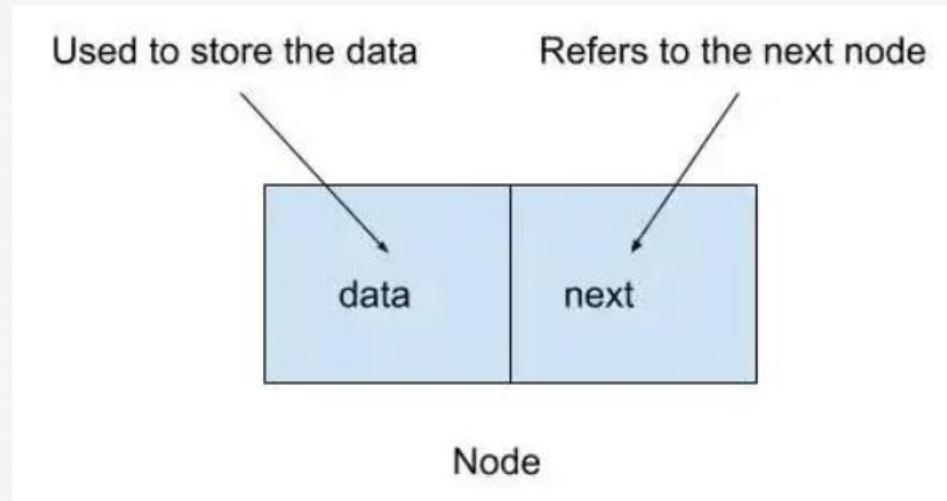
QUÉ ES UN NODO

Es un elemento que almacena: un dato y un enlace hacia otro nodo



QUÉ ES UN NODO

- Un **Node** es un objeto que contiene los atributos **data** y **next**.
- El atributo **data** almacena los **datos**.
- El atributo **next** se refiere al siguiente **nodo**



QUÉ ES UN NODO

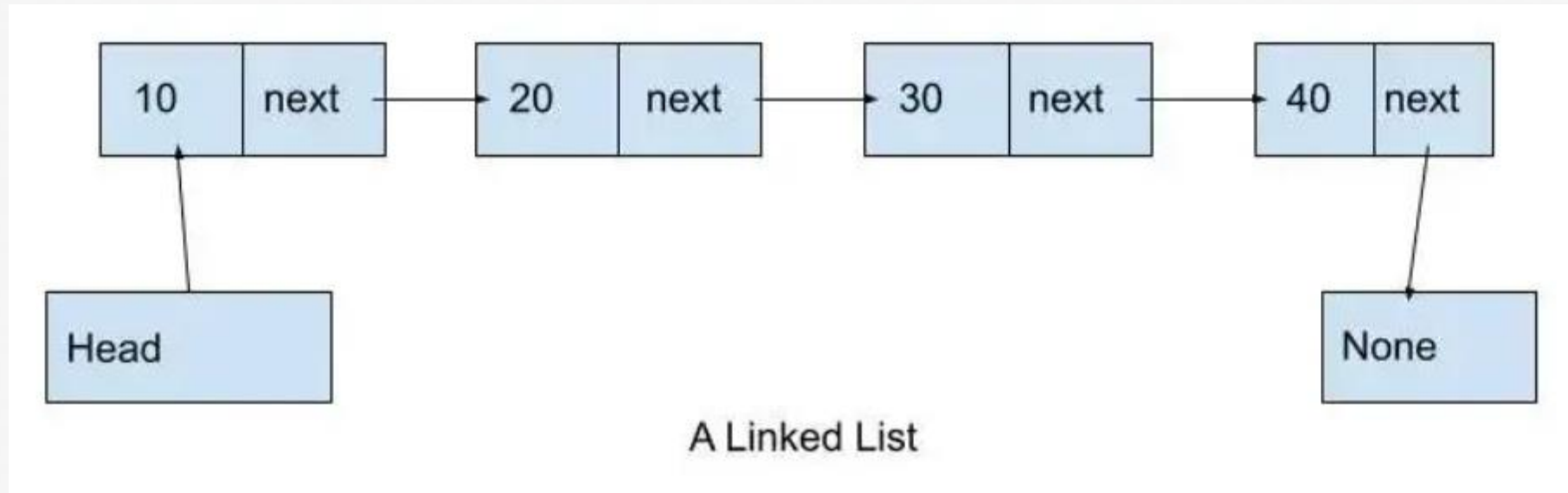
Para programar un nodo en Python usamos los conceptos de programación orientada a objetos

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

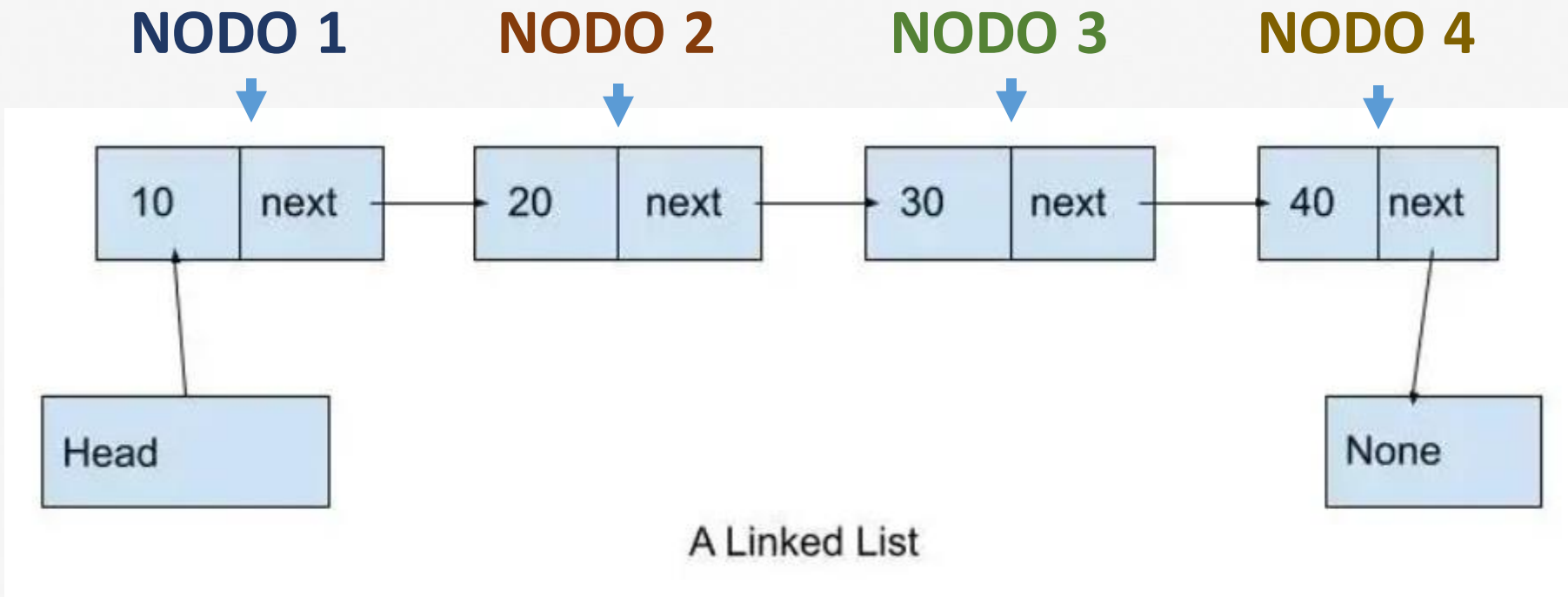
```
myNode = Node(10)
print("El dato en el nodo es:", myNode.data)
print("El siguiente nodo es:", myNode.next)
```


LISTAS ENLAZADAS

Lista enlazada es una estructura de datos que consta de ***nodos conectados***, donde cada nodo contiene un valor y una referencia al siguiente nodo.

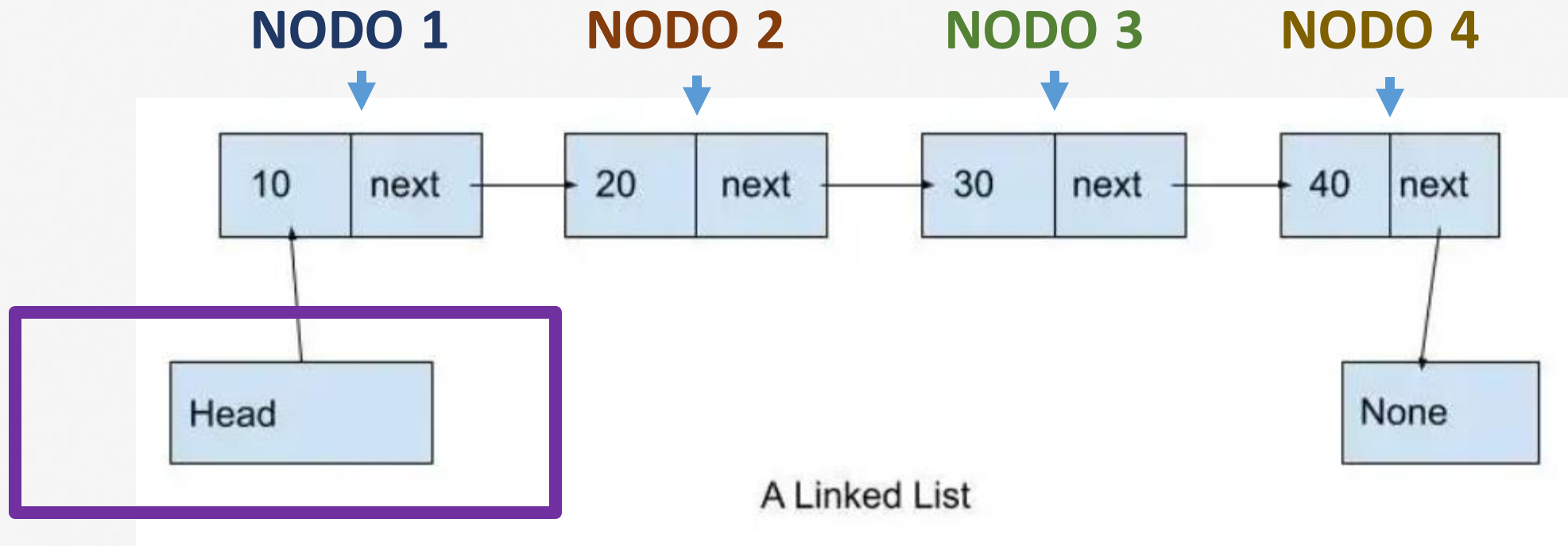


LISTAS ENLAZADAS



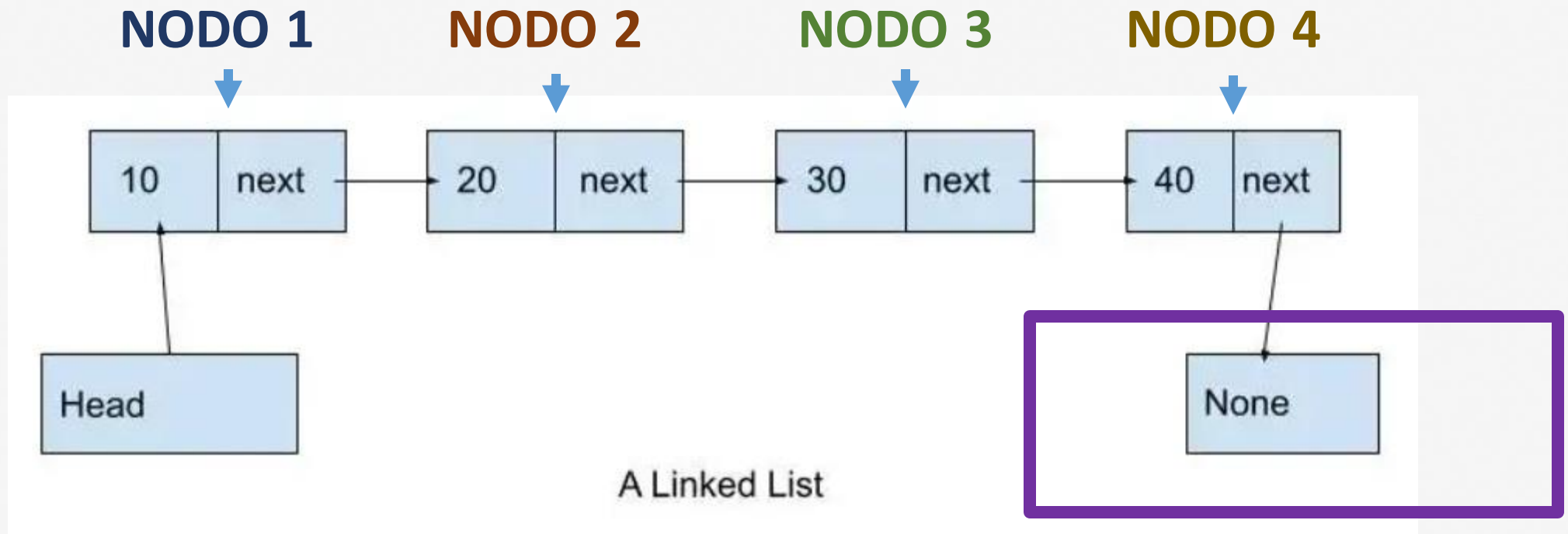
LISTAS ENLAZADAS

- **Head:** es una variable que hace referencia al primer nodo



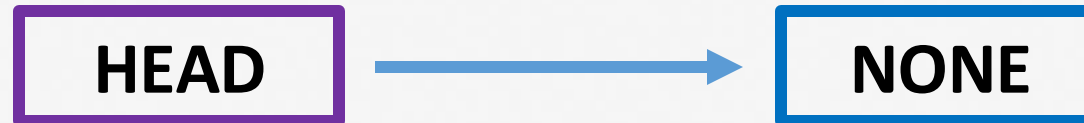
LISTAS ENLAZADAS

- **Tail:** es una variable que apunta al final de la lista y es NONE indicando que no sigue ningún otro nodo



LISTAS ENLAZADAS

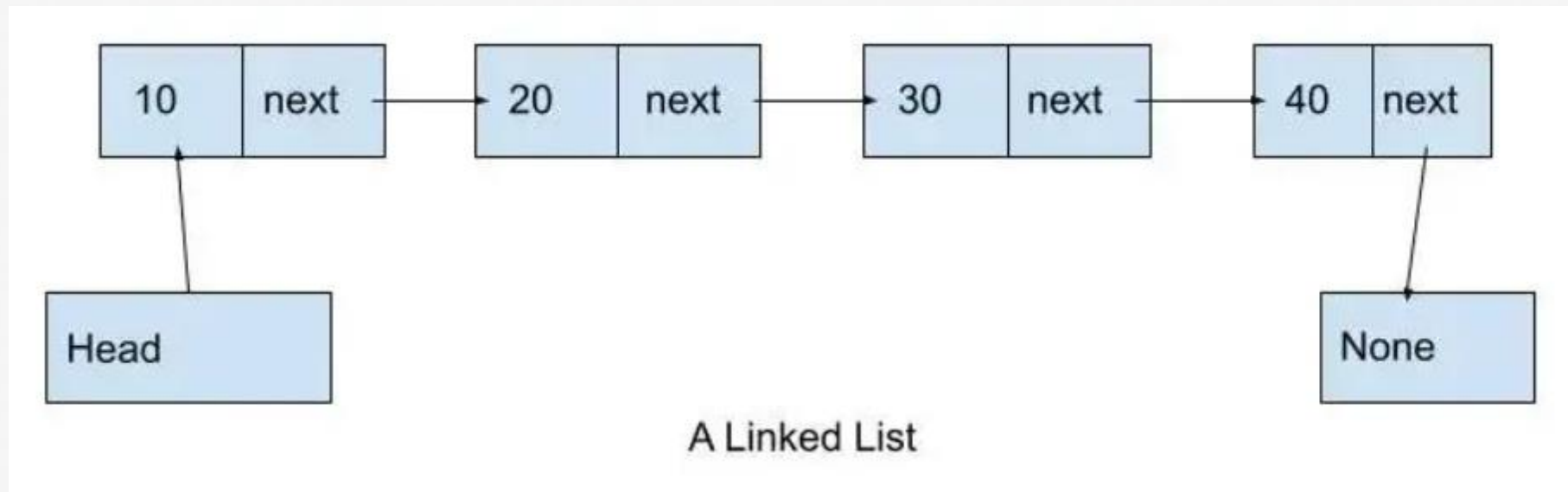
Si una lista enlazada está vacía, la variable *Head* se referirá al objeto None.



LISTAS ENLAZADAS

Para crear una lista enlazada, partimos de la creación de un *nodo*

En el atributo *next* de ese primer nodo, anidamos el nodo *siguiente*



LISTAS ENLAZADAS

Para crear una lista enlazada, partimos de la creación de un *nodo*

En el atributo *next* de ese primer nodo, anidamos el nodo *siguiente*

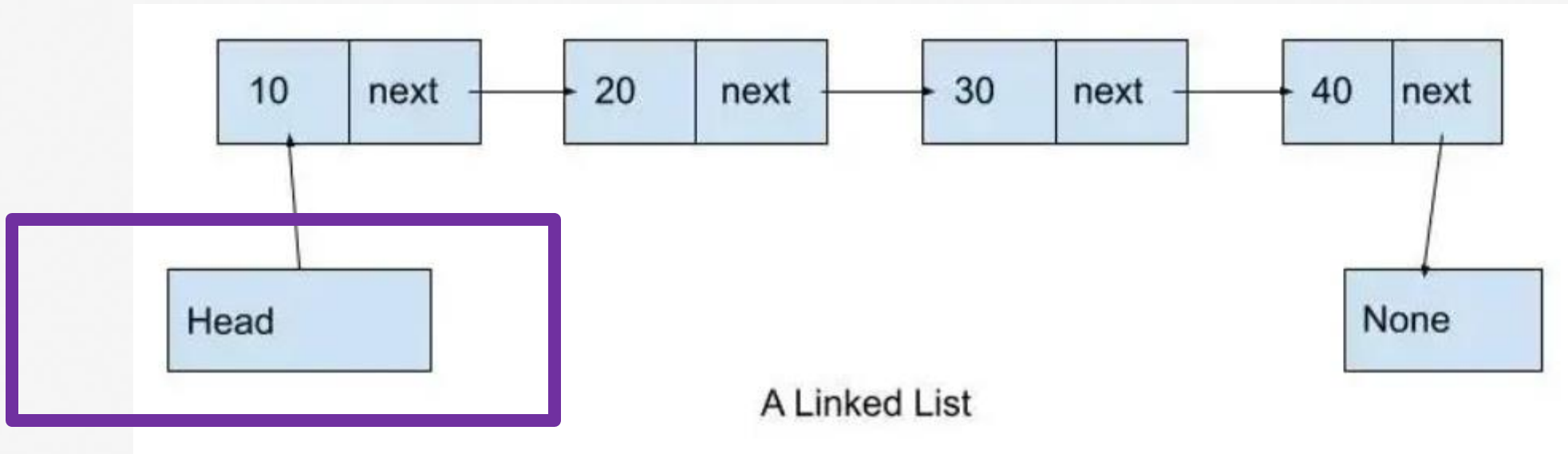
```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```



```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

LISTAS ENLAZADAS

Cuidado ! : antes de empezar a enlazar nodos, debemos definir el Head de la lista enlazada



LISTAS ENLAZADAS

Cuidado ! : antes de empezar a enlazar nodos, debemos definir el Head de la lista enlazada

El Head es **None** por defecto, indicando que la lista enlazada *inicia vacía*

```
class LinkedList:  
    def __init__(self):  
        self.Head = None
```


LISTAS ENLAZADAS

Qué temenos hasta ahora:

- La clase nodo
- La clase lista enlazada

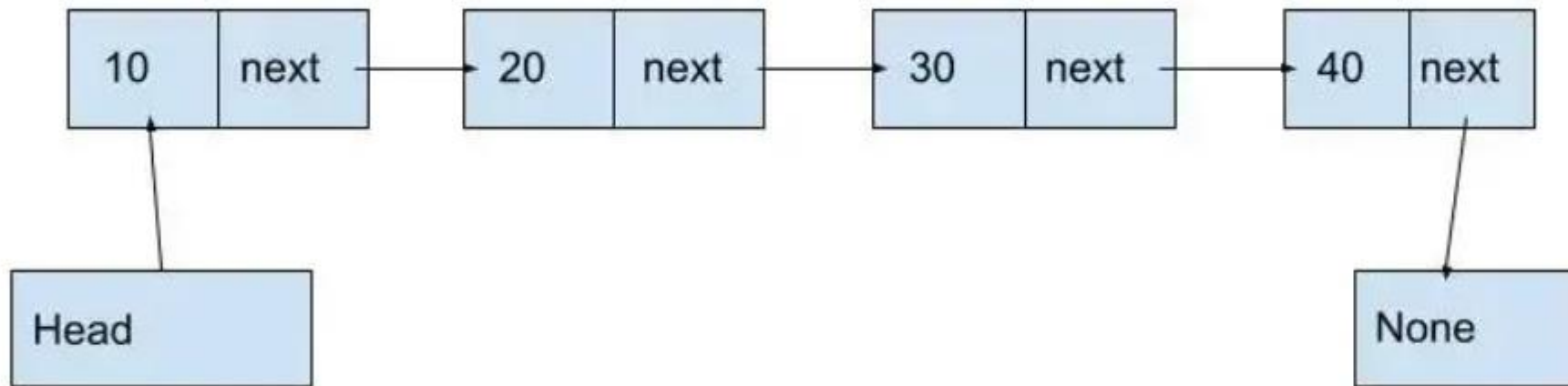
Usando nodos,
Empezaremos a llenar la lista enlazada

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

```
class LinkedList:  
    def __init__(self):  
        self.Head = None
```

LISTAS ENLAZADAS

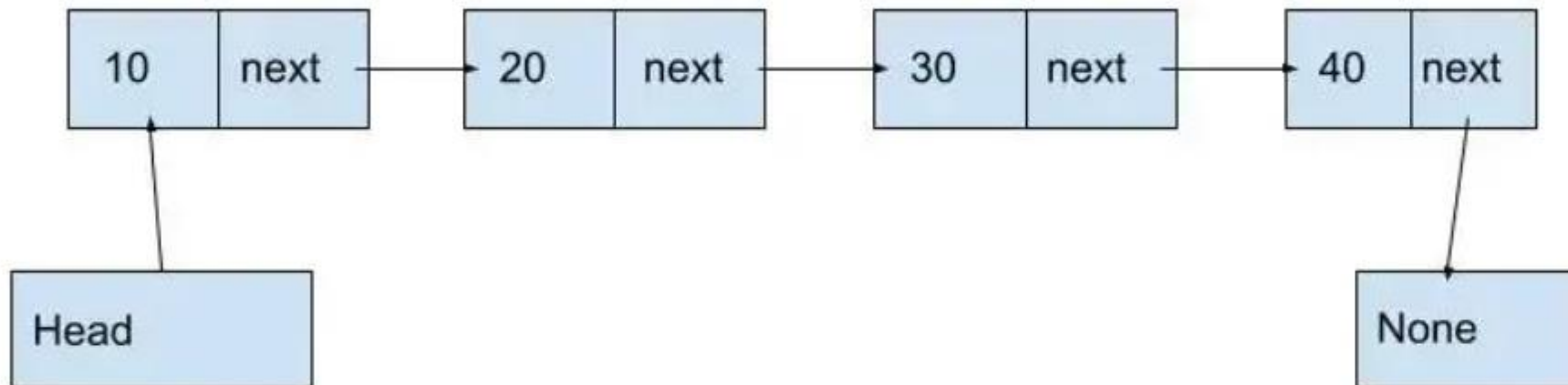
Para crear la lista del ejemplo, primero creamos un objeto de la clase LinkedList



A Linked List

LISTAS ENLAZADAS

```
myLinkedList = LinkedList()
```

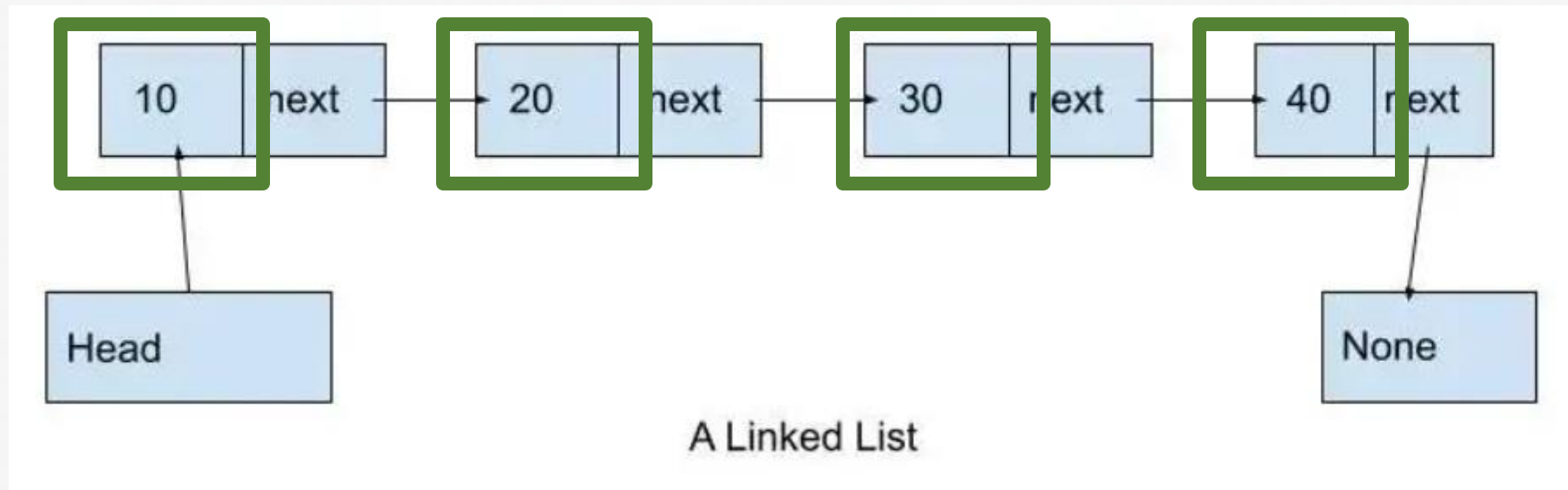


A Linked List

LISTAS ENLAZADAS

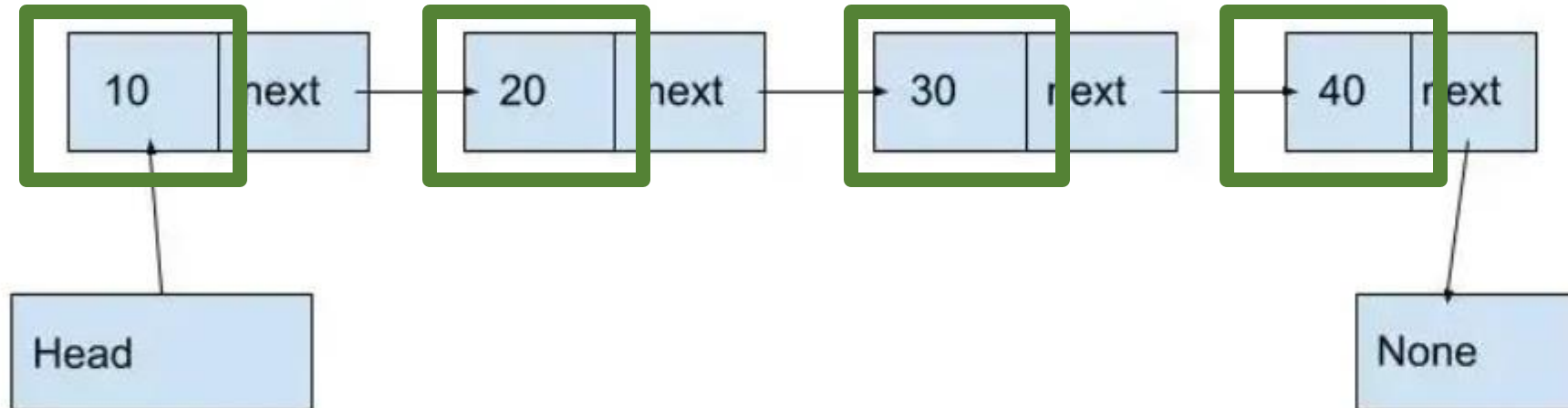
Ahora definimos los cuatro nodos que almacenan: 10 , 20 , 30 y 40

(por ahora, los nodos NO están interconectados)



LISTAS ENLAZADAS

```
myNode1 = Node(10)  
myNode2 = Node(20)  
myNode3 = Node(30)  
myNode4 = Node(40)
```

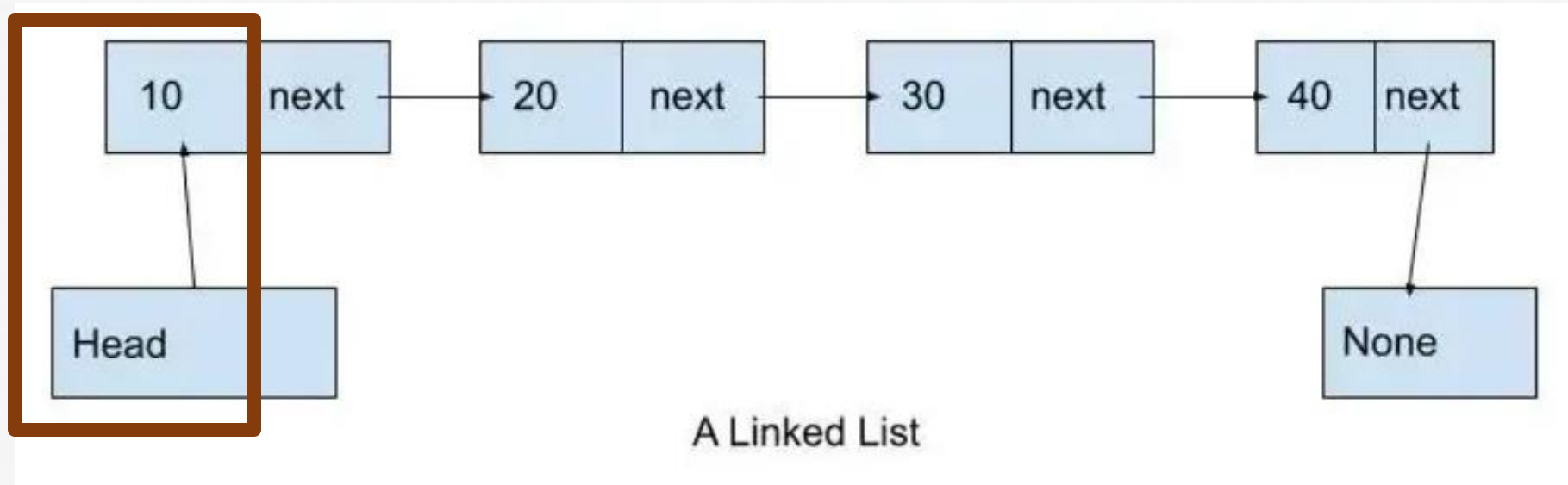


A Linked List

LISTAS ENLAZADAS

Ya tenemos el objeto lista y los nodos, quién será entonces el **Head** de la lista?

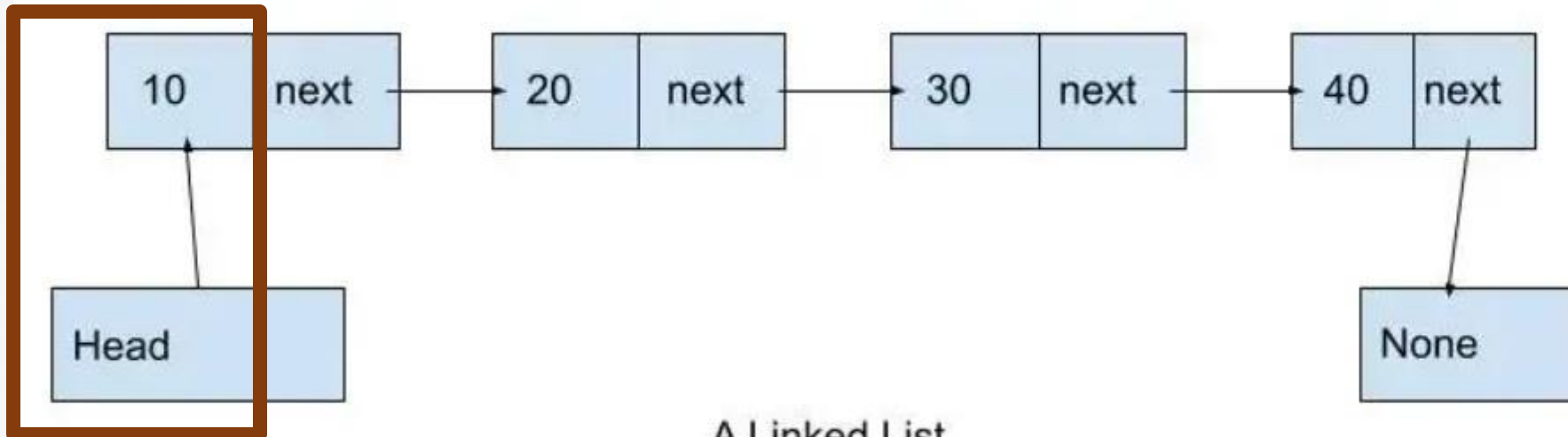
Debemos definir el **nodo** que será el **primer** elemento de la lista



LISTAS ENLAZADAS

```
myLinkedList.Head = myNode1
```

```
myNode1 = Node(10)  
myNode2 = Node(20)  
myNode3 = Node(30)  
myNode4 = Node(40)
```

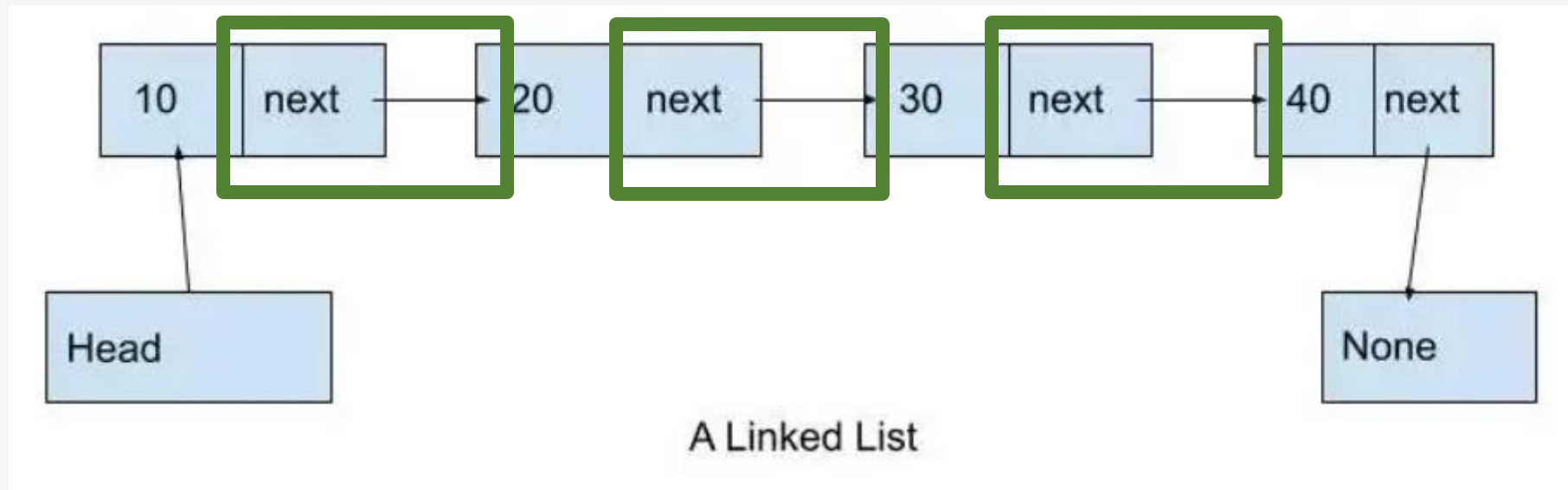


A Linked List

LISTAS ENLAZADAS

Ya tenemos el objeto lista , los nodos, y el Head

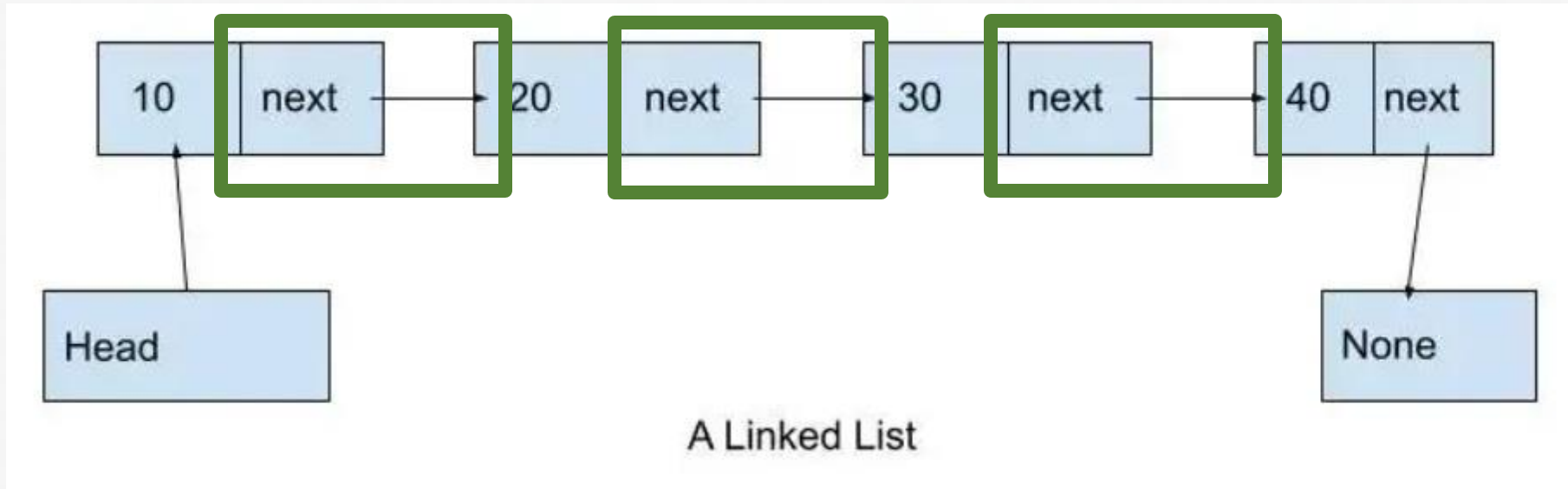
Ahora nos falta interconectar los nodos entre sí



LISTAS ENLAZADAS

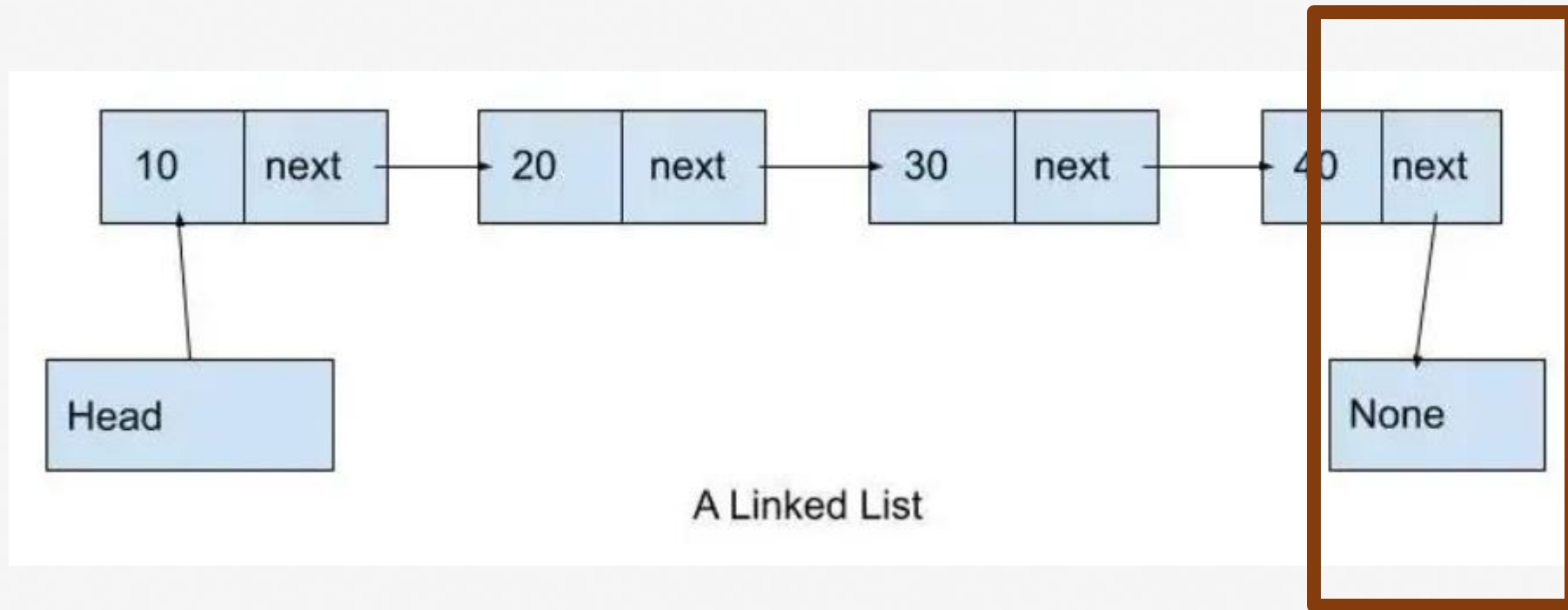
```
myNode1.next = myNode2  
myNode2.next = myNode3  
myNode3.next = myNode4
```

```
myNode1 = Node(10)  
myNode2 = Node(20)  
myNode3 = Node(30)  
myNode4 = Node(40)
```



LISTAS ENLAZADAS

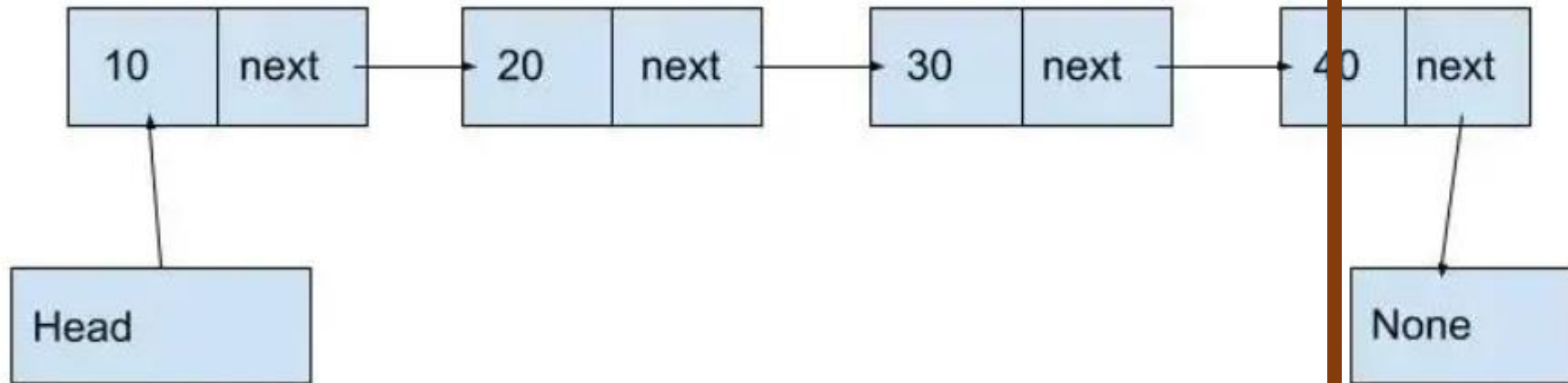
Nota ! : Recuerden que el atributo next del último nodo es None
Indicando que es el final de la lista (tail)



LISTAS ENLAZADAS

```
print(myNode4.next)
```

None

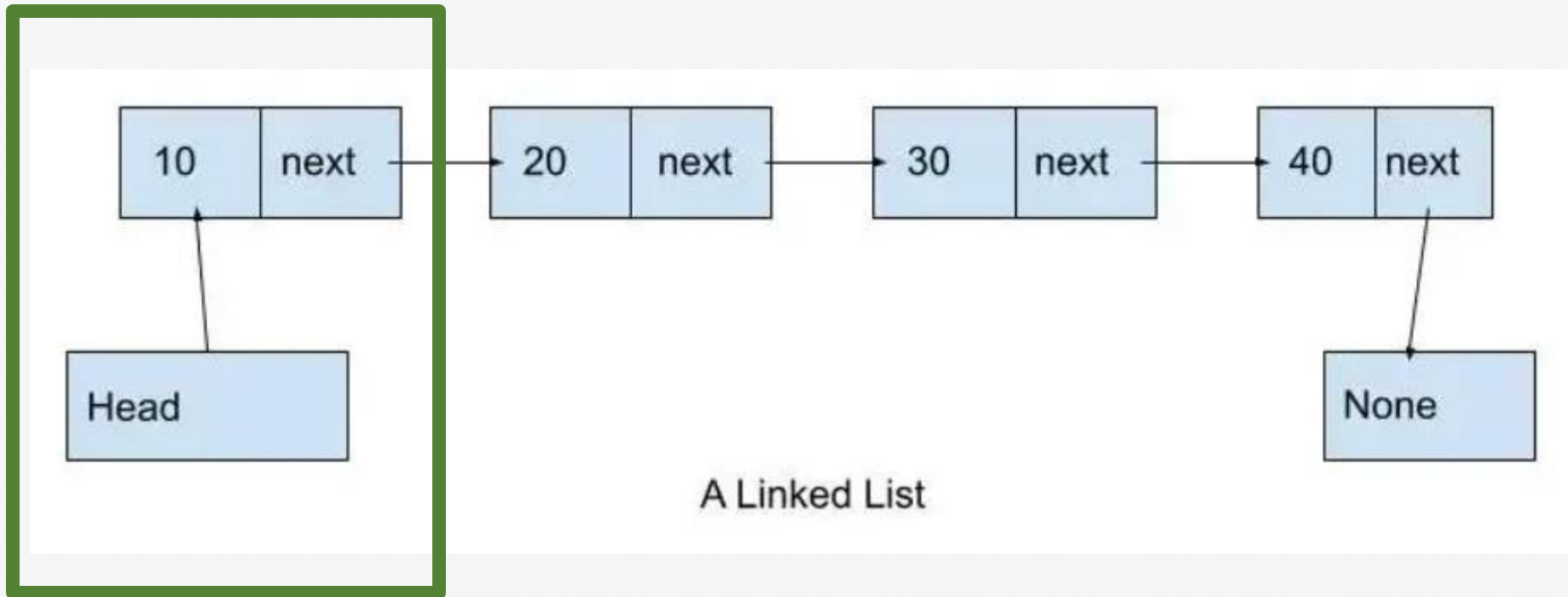


A Linked List

LISTAS ENLAZADAS

Cómo acceder a los elementos de la lista enlazada

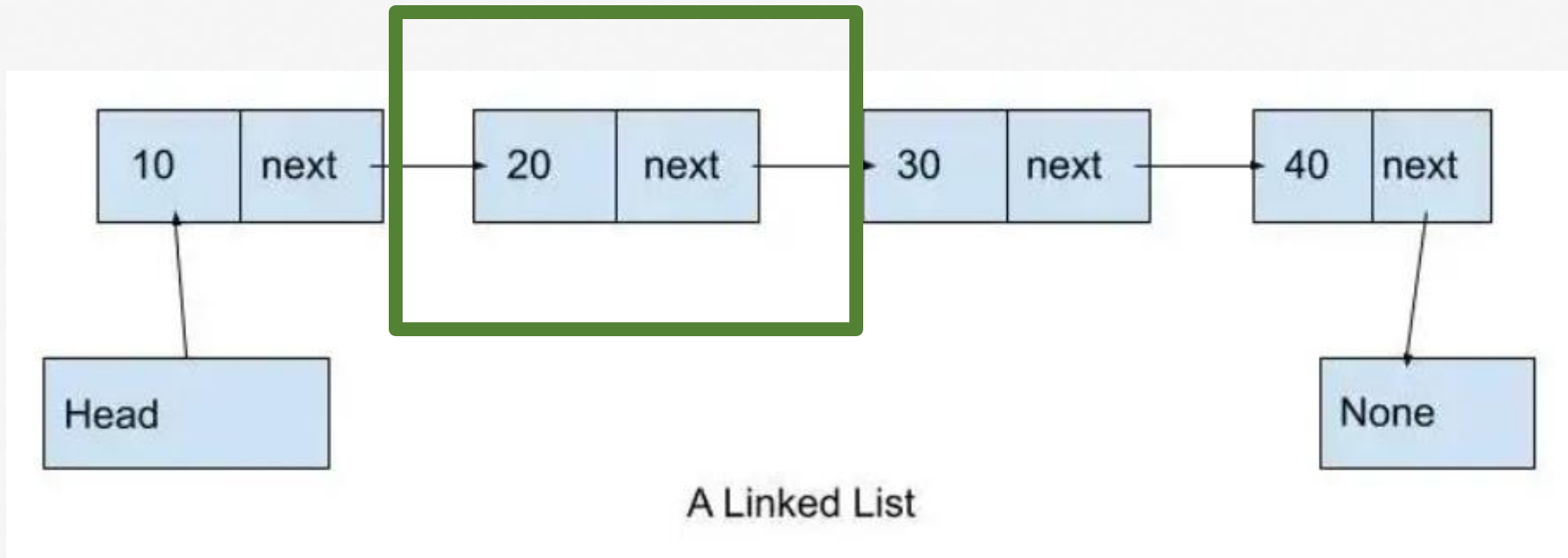
```
print(myLinkedList.Head.data)
```



LISTAS ENLAZADAS

Cómo acceder a los elementos de la lista enlazada

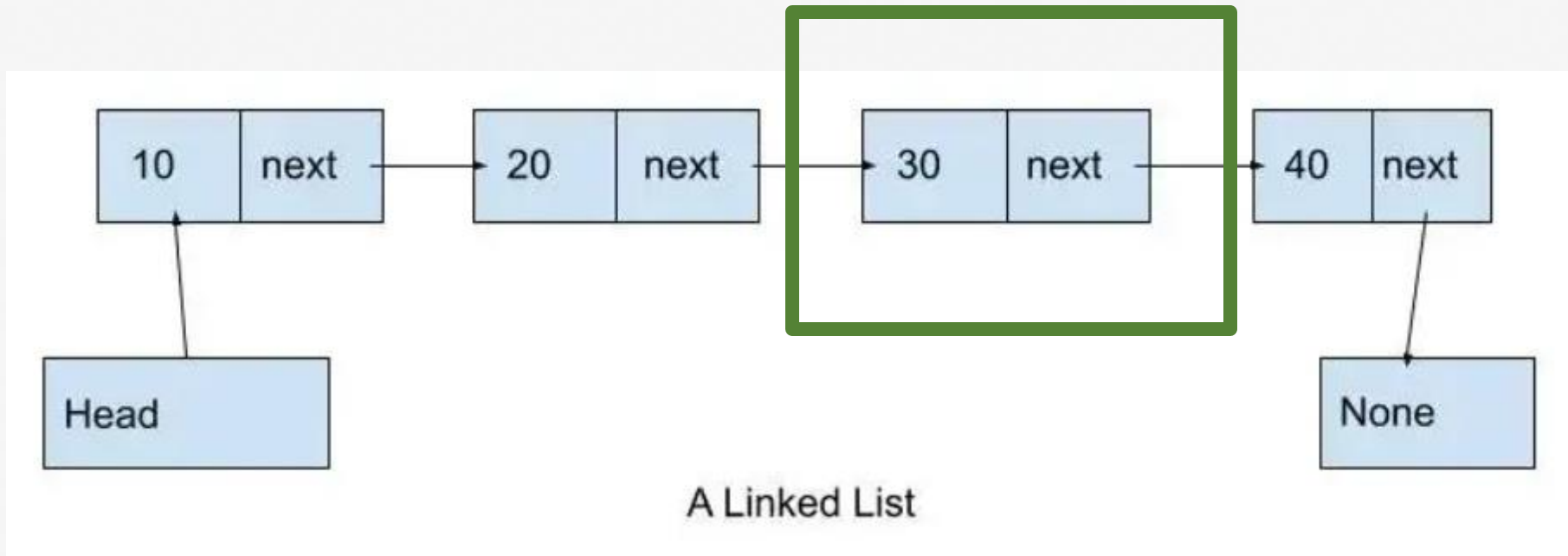
```
print(myLinkedList.Head.next.data)
```



LISTAS ENLAZADAS

Cómo acceder a los elementos de la lista enlazada

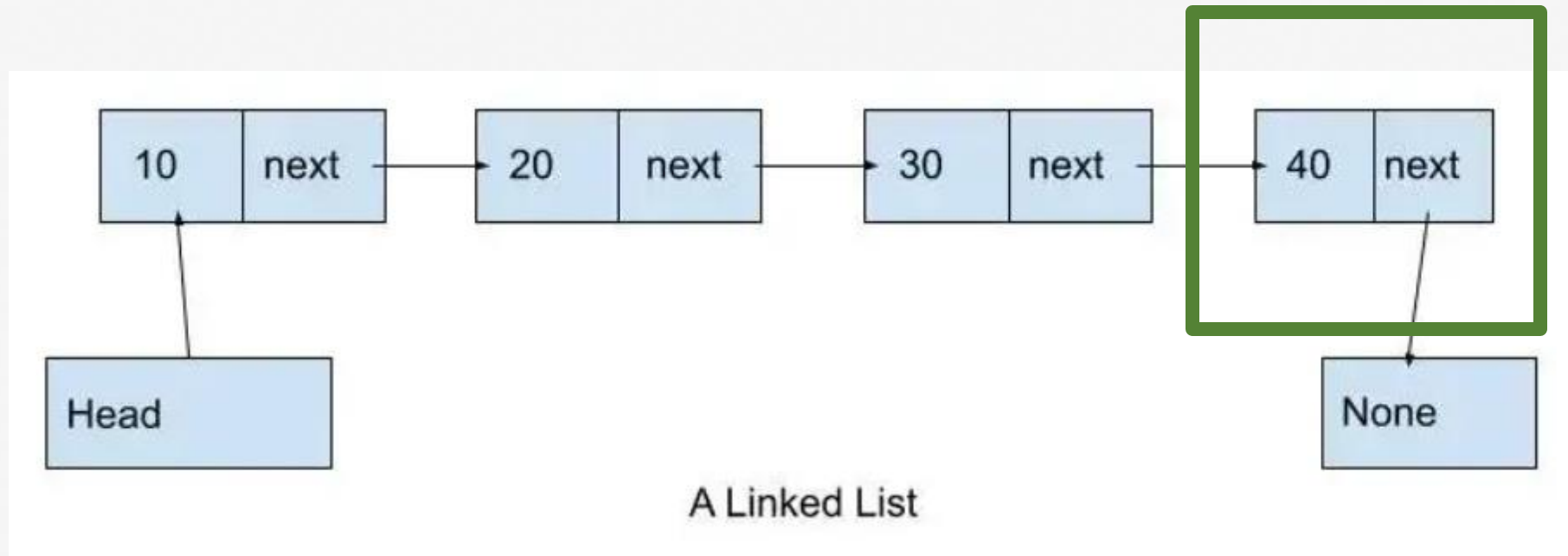
```
print(myLinkedList.Head.next.next.data)
```



LISTAS ENLAZADAS

Cómo acceder a los elementos de la lista enlazada

```
print(myLinkedList.Head.next.next.next.data)
```





Institución
Universitaria
Reacreditada en Alta Calidad

LISTAS ENLAZADAS - MÉTODOS

Las listas enlazadas tienen definidos diferentes métodos que permiten acceder y manipular los datos almacenados en ellas

LISTAS ENLAZADAS - MÉTODOS

- Método para imprimir todos los elementos de una lista enlazada:

```
class LinkedList:  
    def __init__(self):  
        self.Head = None
```

```
    def printList(self):  
        current = self.Head  
        while current is not None:  
            print(current.data, end=" ")  
            current = current.next
```

Comenzamos desde el
elemento HEAD

Imprimimos el dato
almacenado en el elemento

Nos movemos hacia el
elemento next

LISTAS ENLAZADAS - MÉTODOS

- Método para imprimir todos los elementos de una lista enlazada:

```
myLinkedList = LinkedList()  
myNode1 = Node(10)  
myNode2 = Node(20)  
myNode3 = Node(30)  
myNode4 = Node(40)  
myLinkedList.Head = myNode1  
myNode1.next = myNode2  
myNode2.next = myNode3  
myNode3.next = myNode4  
  
myLinkedList.printList()
```

10 20 30 40

LISTAS ENLAZADAS - MÉTODOS

Cómo Podemos insertar elementos nuevos en una lista?

Hay cuatro situaciones al insertar un elemento en una lista enlazada:

- La lista enlazada puede estar **vacía** antes de la inserción
- Tenemos que insertar un elemento al **principio** de una lista enlazada no vacía
- Tenemos que insertar un elemento al **final** de una lista enlazada
- Tenemos que insertar un elemento en una **posición** dada en la lista enlazada

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos en una lista enlazada vacía

Definimos el método *insertIntoEmptyList()* para hacer esto, como se trata de una lista *vacía*, el elemento a guardar deberá ser el *HEAD*.

El método recibe un elemento con el que se crea un nuevo *nodo*

```
def insertIntoEmptyList(self, element):  
    newNode = Node(element)  
    self.Head = newNode
```

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos en una lista enlazada vacía

```
class LinkedList:
    def __init__(self):
        self.Head = None

    def printList(self):
        current = self.Head
        while current is not None:
            print(current.data, end=" ")
            current = current.next

    def insertIntoEmptyList(self, element):
        newNode = Node(element)
        self.Head = newNode
```

```
myLinkedList = LinkedList()
myLinkedList.insertIntoEmptyList(10)
myLinkedList.printList()
```

10

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al inicio de una lista

Para insertar un elemento al comienzo de una lista *no vacía*, definiremos un método *insertAtBeginning()* que toma un elemento como entrada y lo agrega al comienzo de la lista enlazada.

Si añadimos un elemento al inicio de la lista, este se debe convertir en el nuevo *HEAD*

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al inicio de una lista

Si añadimos un elemento al inicio de la lista, este se debe convertir en el nuevo **HEAD**

HEAD

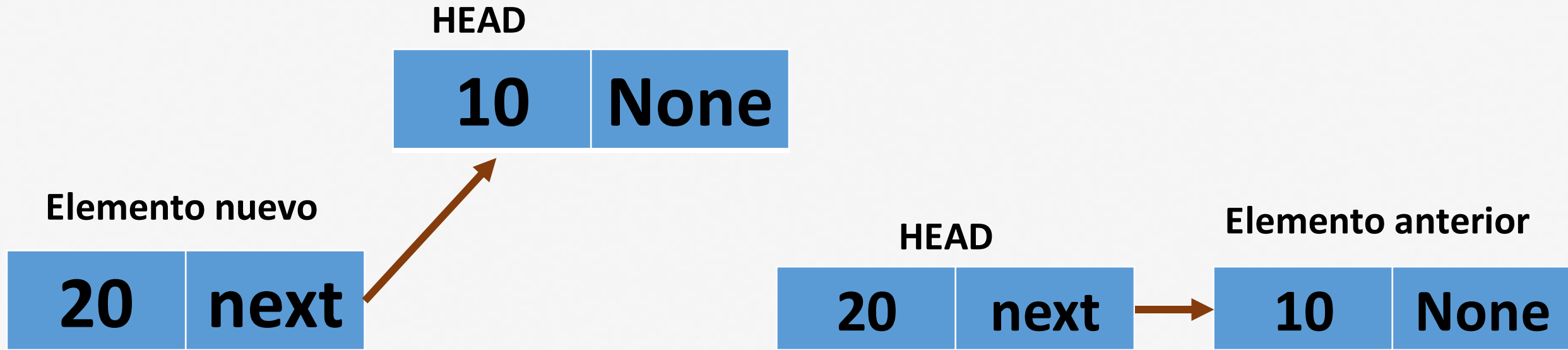
10	next
----	------

Elemento nuevo

20	None
----	------

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al inicio de una lista



LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al inicio de una lista

```
def insertAtBeginning(self, element):  
    newNode = Node(element)  
    newNode.next = self.Head  
    self.Head = newNode
```

Creamos un nodo con
el nuevo elemento

El atributo Next del nuevo
nodo apunta hacia el nodo
anterior

El nuevo nodo reemplaza la
posición del anterior HEAD



LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al inicio de una lista

Nota! Este método desplaza los elementos hacia atrás para poder situar los nuevos elementos adelante

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al inicio de una lista

```
class LinkedList:
    def __init__(self):
        self.Head = None

    def printList(self):
        current = self.Head
        while current is not None:
            print(current.data, end=" ")
            current = current.next

    def insertIntoEmptyList(self, element):
        newNode = Node(element)
        self.Head = newNode

    def insertAtBeginning(self, element):
        newNode = Node(element)
        newNode.next = self.Head
        self.Head = newNode
```

```
myLinkedList = LinkedList()
myLinkedList.insertIntoEmptyList(10)
myLinkedList.insertAtBeginning(20)
myLinkedList.insertAtBeginning(30)
myLinkedList.printList()
```

30 20 10

LISTAS ENLAZADAS - MÉTODOS

Podemos unir los dos métodos de inserción en uno solo

```
def insertAtBeginning(self, element):  
    if self.Head is None:  
        newNode = Node(element)  
        self.Head = newNode  
    else:  
        newNode = Node(element)  
        newNode.next = self.Head  
        self.Head = newNode
```

Si el elemento HEAD está vacío
(None) sitúa el primer elemento
en esa posición

LISTAS ENLAZADAS - MÉTODOS

```
class LinkedList:
    def __init__(self):
        self.Head = None

    def printList(self):
        current = self.Head
        while current is not None:
            print(current.data, end=" ")
            current = current.next

    def insertAtBeginning(self, element):
        if self.Head is None:
            newNode = Node(element)
            self.Head = newNode
        else:
            newNode = Node(element)
            newNode.next = self.Head
            self.Head = newNode
```

```
myLinkedList = LinkedList()
myLinkedList.insertAtBeginning(10)
myLinkedList.insertAtBeginning(20)
myLinkedList.insertAtBeginning(30)
myLinkedList.printList()
```

30 20 10

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al final de una lista

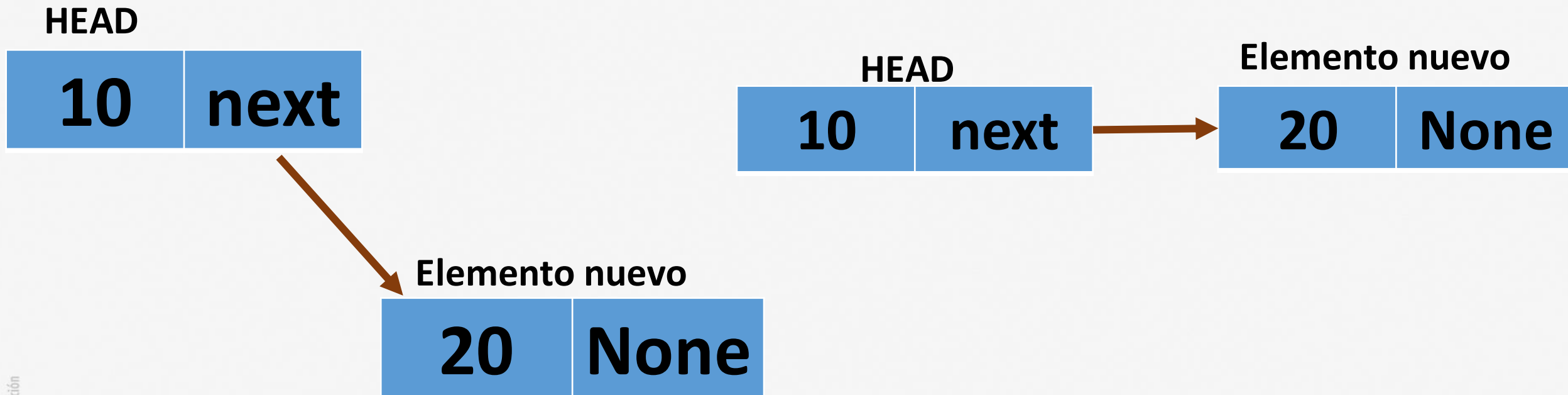
Si la lista está **vacía**, el elemento debe asignarse en el **HEAD**

De lo contrario, debemos recorrer los elementos de la lista hasta llegar al **final** (tail) para **insertar** el nuevo elemento

Recordemos que el **último** elemento de la lista tiene en su atributo next un **None**

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al final de una lista



LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al final de una lista

```
def insertAtEnd(self, element):  
    if self.Head is None:  
        newNode = Node(element)  
        self.Head = newNode  
    else:  
        current = self.Head  
        while current.next is not None:  
            current = current.next  
        newNode = Node(element)  
        current.next = newNode
```

Si la lista está vacía
asignamos el nuevo
elemento en el HEAD

Si la lista no está vacía recorremos todos sus
elementos hasta encontrar el elemento con
el atributo next igual a None

Asignamos el nuevo elemento al atributo
next del último elemento de la lista

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos al final de una lista

```
myLinkedList = LinkedList()  
myLinkedList.insertAtEnd(10)  
myLinkedList.insertAtEnd(20)  
myLinkedList.insertAtEnd(30)  
myLinkedList.printList()
```

10 20 30

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos en una posición dada en una lista

Usaremos una variable de **contador** y un bucle **while** para insertar un elemento en una posición dada en la lista enlazada.

Comenzaremos desde el **Head** y continuaremos moviéndonos al siguiente nodo usando el bucle **while**. En cada iteración, también **incrementaremos** la variable **contador**.

Una vez que alcanzamos el **nodo antes** de la posición dada, salimos del bucle while y hacemos la asignación.

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos en una posición dada en una lista

Nuevo elemento



HEAD



Elemento anterior



Nuevo elemento



HEAD



Elemento anterior



LISTAS ENLAZADAS - MÉTODOS

Insertar elementos en una posición dada en una lista

```
def insertAtPosition(self, element, position):
```

```
    counter = 1
```

```
    current = self.Head
```

```
    while counter < position - 1 and current is not None:
```

```
        counter += 1
```

```
        current = current.next
```

```
    if position == 1:
```

```
        newNode = Node(element)
```

```
        newNode.next = current
```

```
        self.Head = newNode
```

```
    else:
```

```
        newNode = Node(element)
```

```
        newNode.next = current.next
```

```
        current.next = newNode
```

Partimos desde
el HEAD

Recorremos todos los elementos de la
lista hasta encontrar el elemento que se
encuentra una posición anterior a la
posición deseada

Si la posición deseada es la 1 , se añade el
elemento en el HEAD

Se hace la asignación cuando se encuentra
la posición deseada
El next del nuevo elemento apunta hacia
el elemento que ocupa la posición actual

LISTAS ENLAZADAS - MÉTODOS

Insertar elementos en una posición dada en una lista

```
myLinkedList = LinkedList()
myLinkedList.insertAtPosition(10, 1)
print("Los elementos son:")
myLinkedList.printList()
print(" ")
myLinkedList.insertAtPosition(20, 2)
print("Ahora los elementos son:")
myLinkedList.printList()
print(" ")
myLinkedList.insertAtPosition(30, 3)
print("Ahora los elementos son:")
myLinkedList.printList()
print(" ")
myLinkedList.insertAtPosition(40, 2)
print("Ahora los elementos son:")
myLinkedList.printList()
```

LISTAS ENLAZADAS - MÉTODOS

Eliminar un elemento de la lista

Puede haber tres situaciones cuando intentamos eliminar un elemento de una lista enlazada:

- Tenemos que eliminar el **primer** elemento de la lista enlazada
- Tenemos que eliminar el **último** elemento de la lista enlazada
- Tenemos que eliminar el elemento en **cualquier posición** de la lista enlazada

LISTAS ENLAZADAS - MÉTODOS

Eliminar el primer elemento elemento de la lista

Para eliminar el primer elemento de una lista enlazada, primero **comprobaremos** si la lista enlazada está **vacía** o no.

Para ello comprobaremos si el **Head** de la lista enlazada apunta a **None**.

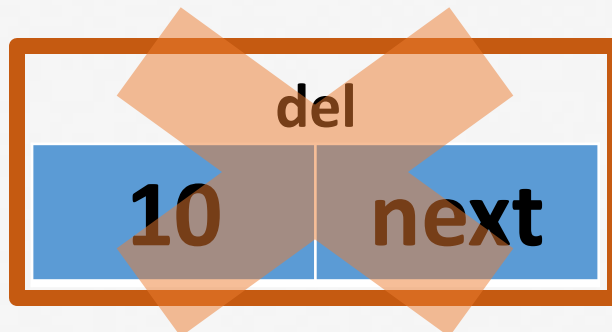
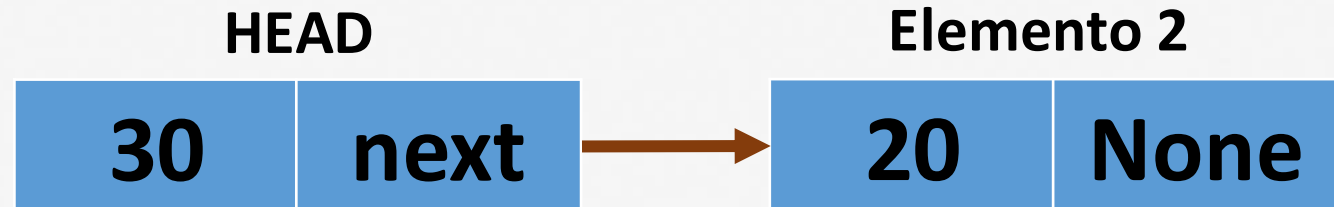
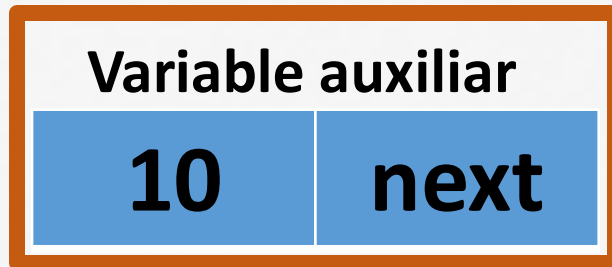
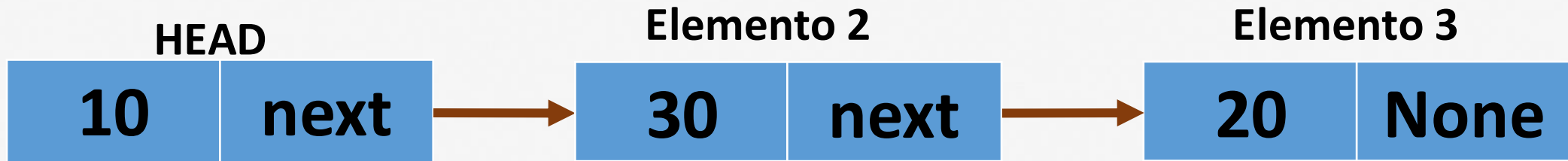
En caso afirmativo, **informaremos** al usuario que la lista vinculada está vacía y no tenemos **ningún** elemento para eliminar.

De lo contrario, asignaremos el **primer** nodo a una variable **temporal**. Después de eso, asignaremos el **segundo** nodo de la lista enlazada al atributo **Head**.

Luego, **eliminaremos** el primer nodo almacenado en la variable temporal usando la instrucción **del**.

LISTAS ENLAZADAS - MÉTODOS

Eliminar el primer elemento elemento de la lista



LISTAS ENLAZADAS - MÉTODOS

Eliminar el primer elemento elemento de la lista

Preguntamos si la lista está vacía

```
def deleteFromBeginning(self):  
    if self.Head is None:  
        print("La lista enlazada está vacía, no se puede eliminar un elemento.")  
        return  
    else:  
        node = self.Head  
        self.Head = self.Head.next  
        del node
```

Almacenamos el HEAD en
una variable auxiliar

Eliminamos el HEAD
almacenado en la variable
auxiliar

El nuevo HEAD será el
segundo elemento de la lista

Eliminar el último elemento de la lista

Para eliminar el primer elemento de una lista enlazada, primero **comprobaremos** si la lista enlazada está **vacía** o no.

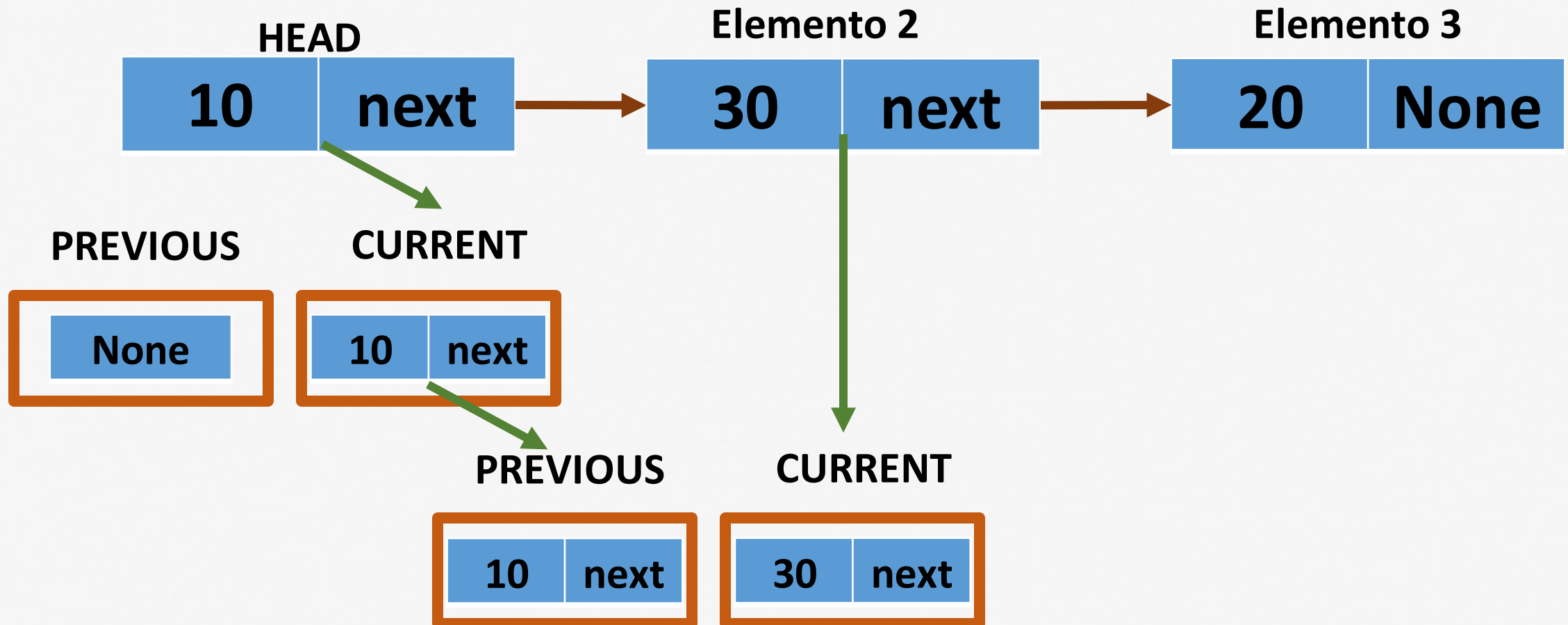
Si hay elementos presentes en la lista, seguiremos el siguiente proceso.

- Asignar el **primer** nodo a una variable **current**.
- Inicializar una variable **previous** a **None**.
- **Recorrer** la lista enlazada utilizando un ciclo while, **asigne** el nodo en la variable **current** a la variable **previous** y avance la variable **current** al **siguiente** nodo hasta que la variable **current** llegue al **último** nodo .
En este caso, el **atributo** next del nodo asignado a **current** pasa a ser **None**.

Una vez que la variable **actual** llegue al **último** nodo, asignaremos **None** al atributo next de la variable **previous** y eliminaremos el nodo asignado a la variable **current**.

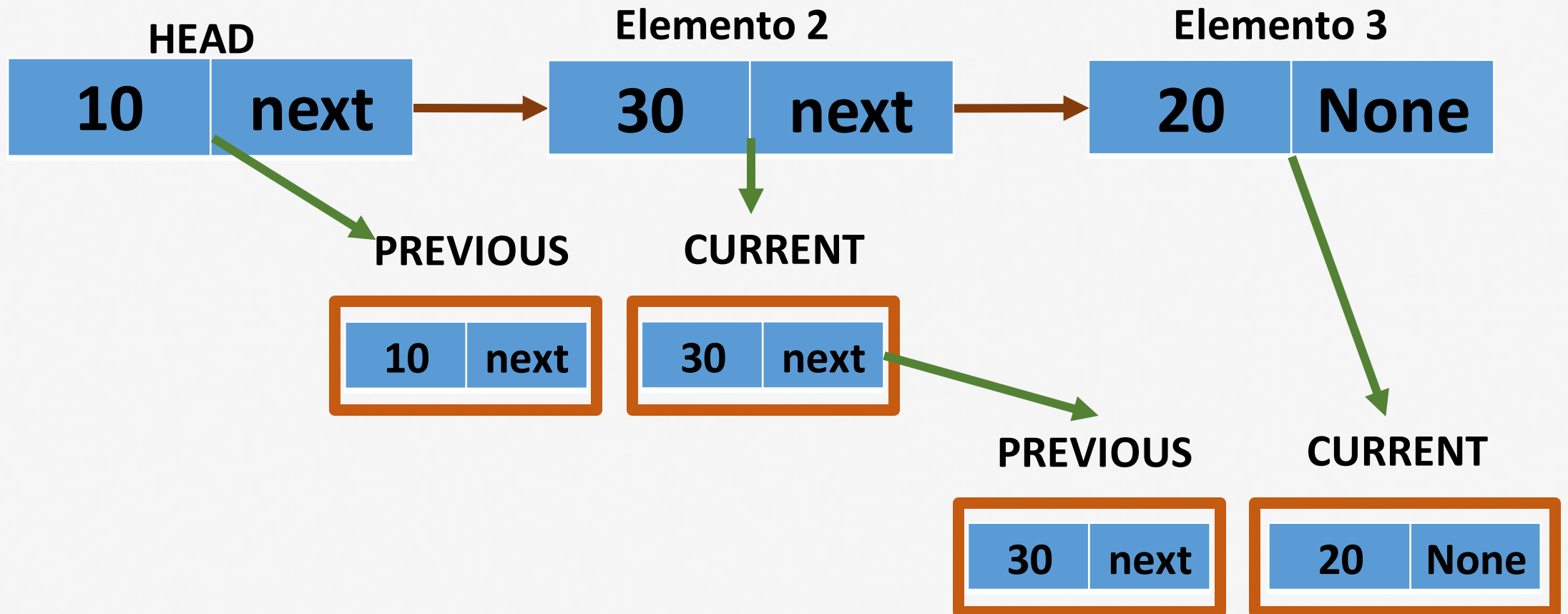
LISTAS ENLAZADAS - MÉTODOS

Eliminar el último elemento de la lista



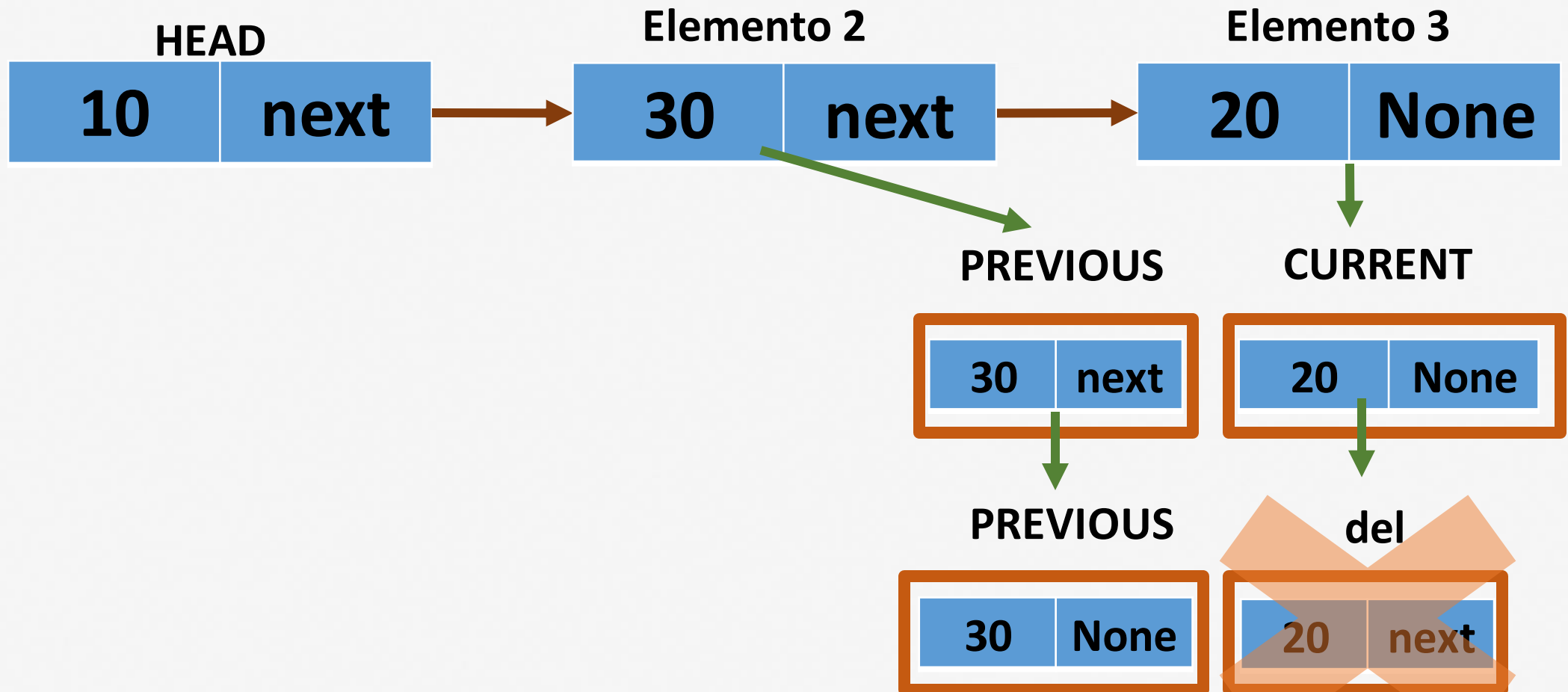
LISTAS ENLAZADAS - MÉTODOS

Eliminar el último elemento de la lista



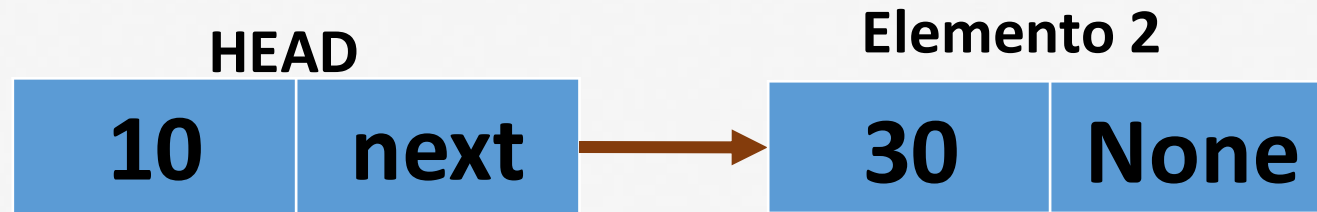
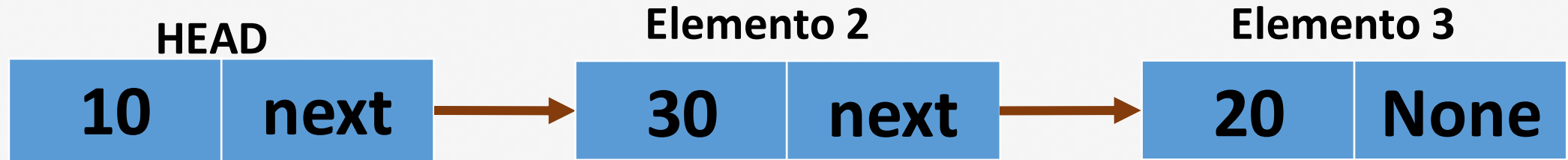
LISTAS ENLAZADAS - MÉTODOS

Eliminar el último elemento de la lista



LISTAS ENLAZADAS - MÉTODOS

Eliminar el último elemento de la lista



LISTAS ENLAZADAS - MÉTODOS

Eliminar el último elemento de la lista

```
def deleteFromLast(self):  
    if self.Head is None:  
        print("La lista enlazada está vacía, no se puede eliminar un elemento.")  
        return  
    else:  
        current = self.Head  
        previous = None  
        while current.next is not None:  
            previous = current  
            current = current.next  
        if previous is not None:  
            previous.next = None  
        else:  
            self.Head = None  
        del current
```

Almacenamos el HEAD en la variable current

Nos movemos por toda la lista buscando el último elemento, actualizando los valores de current y previous

Si solo nos queda un elemento en la lista y lo eliminamos, entonces la lista queda vacía, por lo tanto HEAD es None

LISTAS ENLAZADAS - MÉTODOS

Eliminar el último elemento elemento de la lista

```
myLinkedList.deleteFromLast()  
print("Los elementos de la lista enlazada son:")  
myLinkedList.printList()  
print(" ")  
myLinkedList.deleteFromLast()  
print("Los elementos de la lista enlazada son:")  
myLinkedList.printList()
```

LISTAS ENLAZADAS - MÉTODOS

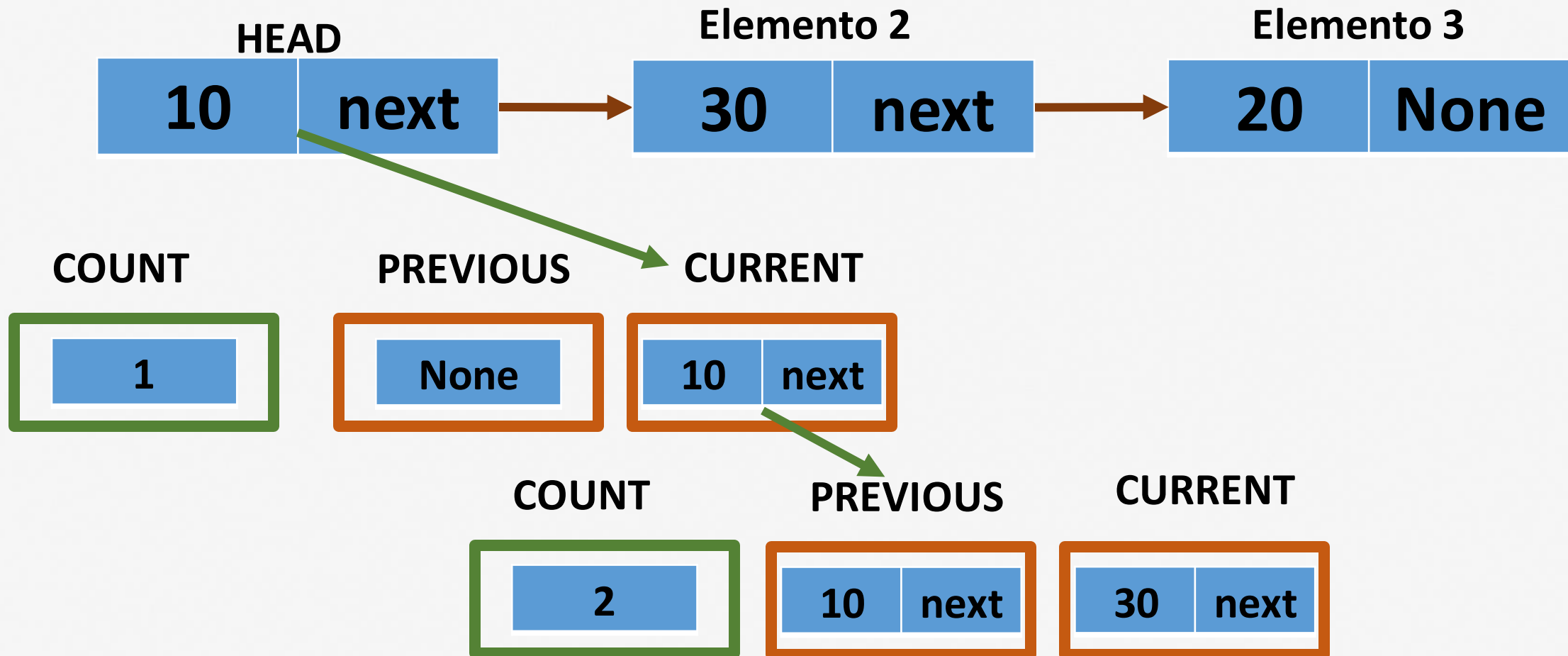
Eliminar un elemento en cualquier posición de la lista

Si hay elementos presentes en la lista enlazada, y tenemos que eliminar un elemento en cualquier otra posición, seguiremos los siguientes pasos

- **Asignar** el primer nodo a una variable **current**
- **Inicializar** una variable **previous** a **None**
- **Inicializar** una variable **count** a 1
- **Recorra** la lista enlazada usando un bucle while, **incremente** count en cada iteración, **asigne** el nodo en la variable **current** a **previous**, y **avance** la variable **current** al siguiente nodo hasta que el **count** tenga el valor de la posición del elemento a borrar o llegamos al final de la lista enlazada

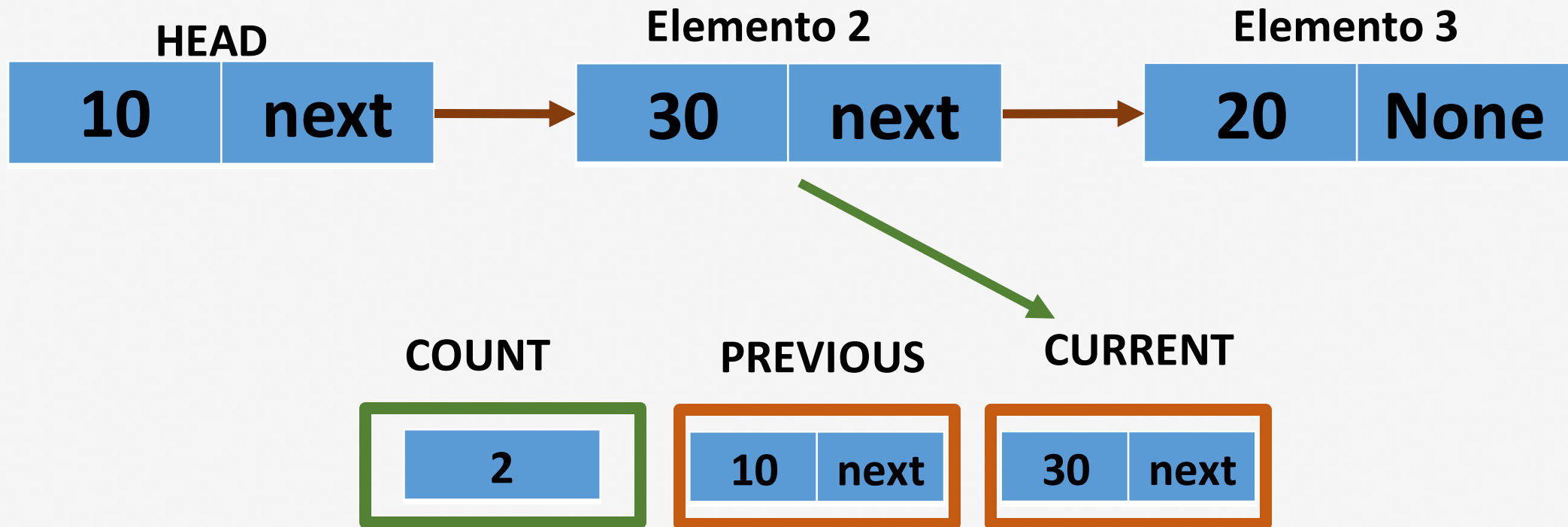
LISTAS ENLAZADAS - MÉTODOS

Eliminar un elemento en cualquier posición de la lista



LISTAS ENLAZADAS - MÉTODOS

Eliminar un elemento en cualquier posición de la lista



LISTAS ENLAZADAS - MÉTODOS

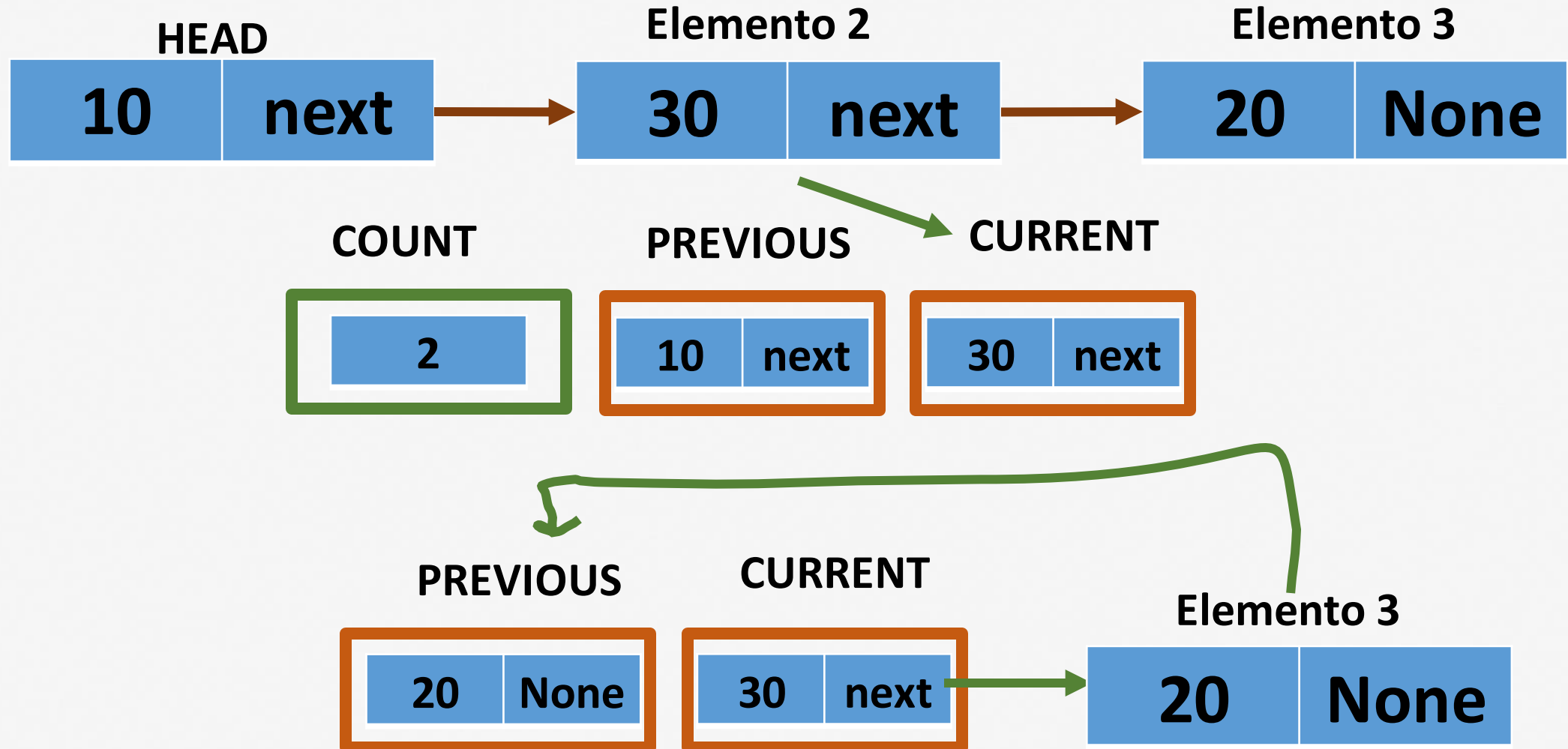
Eliminar un elemento en cualquier posición de la lista

Una vez que el conteo llega a ser igual a la posición del elemento a ser borrado, puede haber dos situaciones:

- Si se quiere eliminar el **HEAD**, entonces actualizamos el **HEAD** con el elemento de la segunda posición
- Si no estamos en el **HEAD**, asignaremos el siguiente nodo de la variable **current** al siguiente atributo del nodo asignado a la variable **previous**. Eliminaremos el nodo asignado a la variable **current**. De esta forma, se eliminará el elemento en la posición dada.

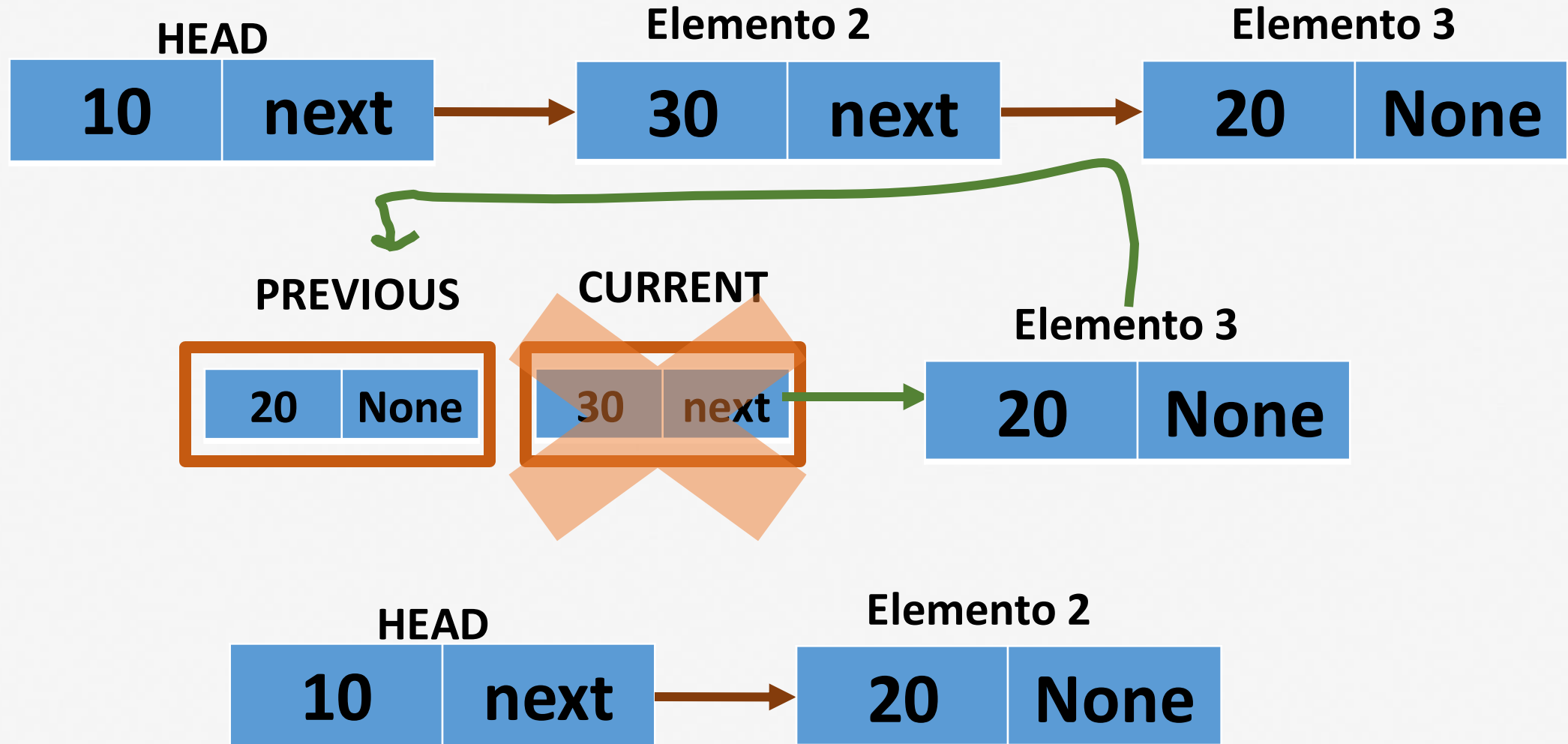
LISTAS ENLAZADAS - MÉTODOS

Eliminar un elemento en cualquier posición de la lista



LISTAS ENLAZADAS - MÉTODOS

Eliminar un elemento en cualquier posición de la lista



LISTAS ENLAZADAS - MÉTODOS

Eliminar un elemento en cualquier posición de la lista

```
def deleteAtPosition(self, position):  
    if self.Head is None:  
        print("La lista enlazada está vacía, no se puede eliminar un elemento.")  
        return  
    else:  
        current = self.Head  
        previous = None  
        count = 1  
        while current.next is not None and count < position:  
            previous = current  
            current = current.next  
            count += 1  
        if current == self.Head:  
            self.Head = current.next  
            del current  
        else:  
            previous.next = current.next  
            del current
```

Comenzamos el recorrido desde el HEAD

Iniciamos el contador con 1

Recorremos la lista hasta llegar a la posición deseada

Eliminamos el elemento de la posición y reconectamos los nodos

LISTAS ENLAZADAS - MÉTODOS

Contar la cantidad de elementos en la lista

Para **contar** el número de **elementos** en una lista enlazada, simplemente **inicializaremos** una variable **count** a 0.

Después de eso, comenzaremos desde el **HEAD** y pasaremos al siguiente nodo usando un **ciclo** while hasta llegar al **final** de la lista enlazada. En cada iteración del bucle while, **incrementaremos** el conteo en 1.

Tras ejecutar el bucle while, tendremos el número de elementos de la lista enlazada en la variable count.

LISTAS ENLAZADAS - MÉTODOS

Contar la cantidad de elementos en la lista

```
def countElements(self):  
    count = 0  
    current = self.Head  
    while current is not None:  
        count += 1  
        current = current.next  
    print("El numero de elementos en la lista es: ", count)
```

LISTAS ENLAZADAS - MÉTODOS

Contar la cantidad de elementos en la lista

```
myLinkedList.printList()  
print(" ")  
myLinkedList.countElements()
```

LISTAS ENLAZADAS - MÉTODOS

Actualizar un nodo en la lista

Puede haber dos situaciones para actualizar el valor en un nodo en la lista enlazada:

- Necesitamos **reemplazar** un valor
- Necesitamos **asignar** un nuevo valor al elemento en cualquier posición dada en la lista enlazada

LISTAS ENLAZADAS - MÉTODOS

Reemplazar el valor de un nodo en la lista

Para **reemplazar** un valor en la lista enlazada, comenzaremos desde el primer nodo y recorreremos la lista enlazada usando un bucle while.

Verificaremos si el nodo **actual** contiene el valor a reemplazar en cada nodo. En caso afirmativo, reemplazaremos el valor en el nodo actual con el nuevo valor.

LISTAS ENLAZADAS - MÉTODOS

Reemplazar el valor de un nodo en la lista

```
def replaceElement(self, old_element, new_element):  
    current = self.Head  
    while current is not None:  
        if current.data == old_element:  
            current.data = new_element  
            break  
        current = current.next
```

LISTAS ENLAZADAS - MÉTODOS

Reemplazar el valor de un nodo en la lista

```
myLinkedList.printList()  
print(" ")  
myLinkedList.replaceElement(30, 100)  
print("Los elementos de la lista enlazada son:")  
myLinkedList.printList()  
print(" ")  
myLinkedList.replaceElement(20, 150)  
print("Los elementos de la lista enlazada son:")  
myLinkedList.printList()  
print(" ")
```


LISTAS ENLAZADAS - MÉTODOS

Actualizar el valor en una posición dada de la lista

Si la lista enlazada está **vacía** y tenemos que **actualizar** algún elemento que **no** sea la primera posición, avisaremos al usuario de que **no se puede hacer**.

Si la lista enlazada está **vacía** y tenemos que actualizar el elemento en la **primera** posición, crearemos un **nuevo** nodo con el elemento dado y asignaremos el nodo al **HEAD** de la lista enlazada.

LISTAS ENLAZADAS - MÉTODOS

Actualizar el valor en una posición dada de la lista

Si la lista enlazada **no** está vacía creamos una variable **contador** que comience en 1.

Después de eso, **recorreremos** la lista enlazada usando un ciclo while. En cada iteración del ciclo while, nos **moveremos** al siguiente nodo en la lista enlazada, **incrementaremos** la variable contador en 1 y comprobaremos si hemos **llegado** a la posición del elemento que necesita ser actualizado.

Si **llegamos** a la posición que necesita ser actualizada, **actualizaremos** el valor en el nodo actual de la lista enlazada y notificaremos al usuario.

Si **no** podemos llegar a la posición que necesita ser actualizada y el bucle while termina, le notificaremos al usuario que no hay suficientes elementos y no podemos actualizar el valor.

LISTAS ENLAZADAS - MÉTODOS

Actualizar el valor en una posición dada de la lista

```
def updateAtPosition(self, new_element, position):  
    if self.Head is None and position != 1:  
        print("No element to update in the linked list.")  
        return  
    elif self.Head is None and position == 1:  
        newNode = Node(new_element)  
        self.Head = newNode  
        return  
    count = 1  
    current = self.Head  
    while current.next is not None and count < position:  
        count += 1  
        current = current.next  
    if count == position:  
        current.data = new_element  
    elif current.next is None:  
        print("No hay tantos elementos en la lista enlazada")
```

LISTAS ENLAZADAS - MÉTODOS

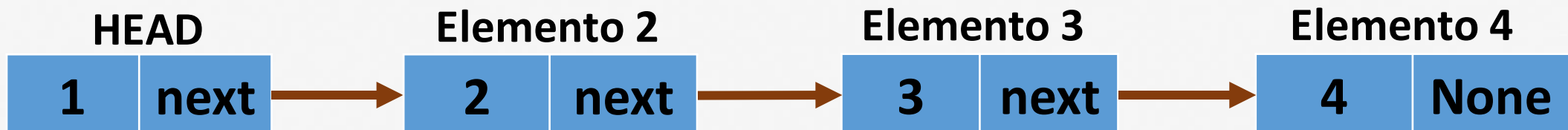
Actualizar el valor en una posición dada de la lista

```
myLinkedList.printList()
print(" ")
myLinkedList.updateAtPosition(100, 3)
print("Los elementos de la lista enlazada son:")
myLinkedList.printList()
print(" ")
myLinkedList.updateAtPosition(150, 12)
print("Los elementos de la lista enlazada son:")
myLinkedList.printList()
print(" ")
```

LISTAS ENLAZADAS - MÉTODOS

Imprimir los elementos de la lista de atrás hacia adelante

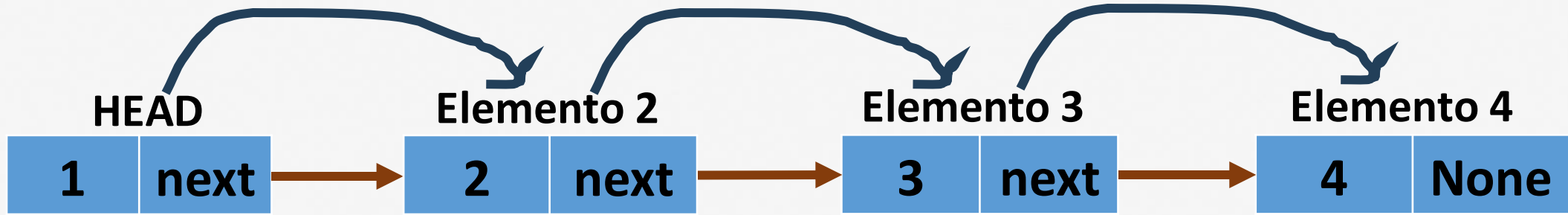
Se puede utilizar una función recursiva que haga un recorrido inverso de la lista enlazada



LISTAS ENLAZADAS - MÉTODOS

Imprimir los elementos de la lista de atrás hacia adelante

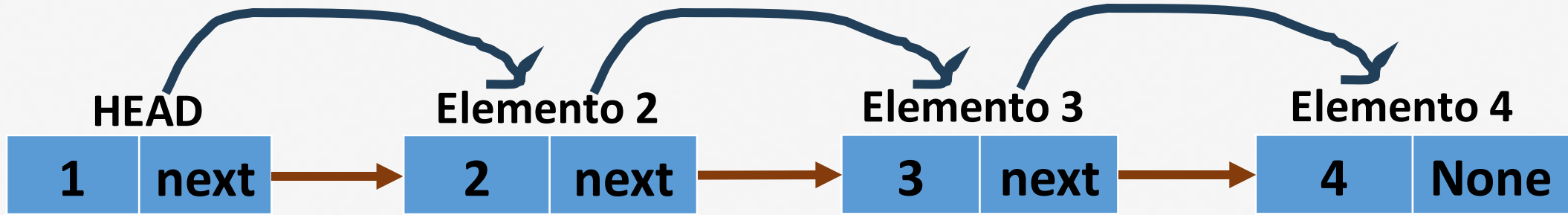
```
def printListReverse(self):  
    self._printListReverseHelper(self.Head)  
  
def _printListReverseHelper(self, node):  
    if node is None:  
        return  
    self._printListReverseHelper(node.next)  
    print(node.data, end=" ")
```



LISTAS ENLAZADAS - MÉTODOS

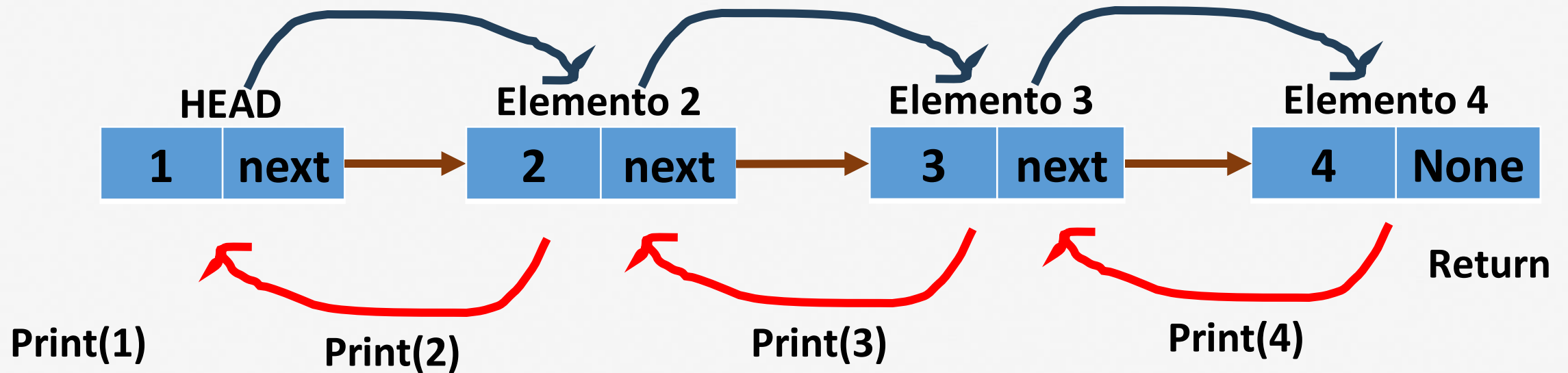
Imprimir los elementos de la lista de atrás hacia adelante

```
_printListReverseHelper(1)  
  _printListReverseHelper(2)  
    _printListReverseHelper(3)  
      _printListReverseHelper(4)
```



LISTAS ENLAZADAS - MÉTODOS

Imprimir los elementos de la lista de atrás hacia adelante



LISTAS ENLAZADAS - MÉTODOS

Imprimir los elementos de la lista de atrás hacia adelante

```
linked_list = LinkedList()
linked_list.insertAtEnd(1)
linked_list.insertAtEnd(2)
linked_list.insertAtEnd(3)
linked_list.insertAtEnd(4)

print("Lista normal:")
linked_list.printList()

print("\nLista en reversa:")
linked_list.printListReverse()
```

A la lista enlazada creada añada los siguientes métodos:

- Un método que imprima el elemento mayor
- Un método que imprima el elemento menor
- Un método que dado un valor me diga si ese valor está dentro de la lista o no
- Un método que imprima la posición en la que se encuentra un valor dado



Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white swoosh that curves around the end of the lines.